UNIVERSIDADE MUNICIPAL DE SÃO CAETANO DO SUL
Bacharelado em Ciência da Computação · Revisão para o ENADE 2026

MATERIAL DE APOIO

Sistemas Operacionais

Material explicativo dos nove tópicos da Matriz de Referência

Squad 6
Jorge Orselli Mutran
Kaick Kenithi Nishiya
Lucas Koiti Zotovici Kobayashi
Lucas Secanho do Prado
Matheus Pestilli Rodrigues João

Como usar este material

Este material foi escrito para sustentar a fala de quem apresenta a revisão. Os slides trazem apenas a síntese e os diagramas; aqui está o desenvolvimento de cada tema — o que dizer sobre cada figura, quais distinções a prova cobra, quais são os erros mais frequentes e o vocabulário que o ENADE usa. Não é um roteiro a ser lido: é o que se precisa entender para falar com segurança.

Cada seção abre com uma introdução que situa os tópicos e diz por que eles importam para a prova. Em seguida, para cada slide, vêm a síntese (o que está projetado), o aprofundamento (o desenvolvimento do tema) e, quando há diagrama, a figura acompanhada de um bloco 'Como ler a figura'. A seção fecha com dois quadros: os termos que a prova usa e os erros mais comuns. O referencial teórico — Tanenbaum, Silberschatz, Machado & Maia, o livro aberto OSTEP, os artigos clássicos e a documentação do kernel Linux — está ao final.

Sumário

Seção 1 — Fundamentos do Sistema Operacional · Tópicos 1, 2 e 3 — visão geral, modos de acesso e arquitetura do núcleo

Seção 2 — Processos, Sincronização e Escalonamento · Tópicos 4 e 5 — PCB, estados, concorrência, deadlock e uso da CPU

Seção 3 — Gerência de Memória Principal · Tópico 6 — hierarquia, particionamento, fragmentação e relocação

Seção 4 — Memória Virtual · Tópico 7 — paginação, segmentação, TLB, substituição e thrashing

Seção 5 — Sistemas de Arquivos e Entrada/Saída · Tópicos 8 e 9 — organização em disco, diretórios, subsistema de E/S e proteção

Seção 1 — Fundamentos do Sistema Operacional

Tópicos 1, 2 e 3 — visão geral, modos de acesso e arquitetura do núcleo

- T1 · Visão geral, máquina de níveis e softwares tradutores
- T2 · Modo usuário/kernel, interrupções, exceções e chamadas de sistema
- T3 · Arquiteturas de núcleo, boot e virtualização

Esta seção cobre os três primeiros tópicos da Matriz de Referência e forma a base conceitual de tudo o que vem depois. O ENADE raramente pergunta 'o que é um sistema operacional' de forma direta; o que ele faz é embutir esses conceitos em situações — um programa que é encerrado ao tentar acessar o hardware, um servidor que não inicializa, uma decisão entre virtualizar e conteinerizar. Quem domina o vocabulário desta seção (modo de acesso, trap, chamada de sistema, núcleo, hipervisor) lê essas situações com rapidez; quem não domina perde tempo decodificando o enunciado.

Os três tópicos se encadeiam: primeiro entendemos o que o SO é e onde ele fica na pilha de abstrações (T1); depois, como o hardware protege o SO dos programas e como os programas pedem serviços ao SO (T2); por fim, como o núcleo é organizado internamente, como ele chega a estar em execução e como se pode rodar mais de um sistema na mesma máquina (T3).

Duas visões do mesmo sistema

Tópico 1 · o SO visto de cima para baixo e de baixo para cima (Tanenbaum)

Síntese projetada

Máquina estendida — de cima para baixo — abstração do hardware: a aplicação chama write(); quem fala com o controlador é o driver, dentro do SO.

Gerenciador de recursos — de baixo para cima — arbitra CPU, memória e dispositivos entre programas concorrentes, por multiplexação no tempo e no espaço.

Aprofundamento

A definição mais repetida em prova é a de Tanenbaum: o sistema operacional é, ao mesmo tempo, uma máquina estendida e um gerenciador de recursos. Atenção à palavra: são duas VISÕES, não duas funções. Tanenbaum não diz que o SO faz duas coisas separadas; diz que o mesmo sistema pode ser olhado de dois ângulos — de cima para baixo e de baixo para cima. É conteúdo canônico de toda bibliografia de SO e, por isso, cobrável em qualquer prova; uma questão bem construída tentaria justamente derrubar a leitura de que se trata de duas tarefas independentes.

A visão de máquina estendida (ou máquina virtual, no sentido antigo do termo) é a de cima para baixo, e é a visão de quem ESCREVE APLICAÇÕES — não a do usuário final, que vê shell, janelas e ícones, coisas que ficam acima do SO. Quem programa não quer saber como funciona um controlador NVMe, quer abrir um arquivo e escrever nele; chama write() e quem de fato conversa com o controlador é o driver, que faz parte do próprio SO. O SO oferece abstrações — arquivo, processo, thread, socket, diretório — que escondem a complexidade do hardware por trás de uma interface estável. Um mesmo programa em C roda em máquinas com discos, controladores e memórias completamente diferentes justamente porque conversa com o SO, e não com o hardware.

A visão de gerenciador de recursos é a de baixo para cima: existem muitos programas e poucos recursos (uma CPU, alguns gigabytes de RAM, um disco, uma impressora), e alguém precisa arbitrar quem usa o quê, quando, por quanto tempo e com que proteção contra os demais. Tanenbaum distingue dois tipos de multiplexação: no tempo (a CPU alterna entre processos; a impressora atende um trabalho por vez) e no espaço (a memória e o disco são divididos entre vários processos simultaneamente).

Um jeito prático de fixar a distinção: a máquina estendida responde 'o que o SO oferece a quem escreve aplicações' (a API de chamadas de sistema); o gerenciador de recursos responde 'o que o SO faz para que vários programas convivam' (a visão de quem projeta o SO). Quando o enunciado fala em esconder detalhes, oferecer abstrações ou simplificar a programação, está na primeira visão. Quando fala em arbitrar, proteger, contabilizar ou compartilhar, está na segunda.

Máquina de níveis (hierarquia de abstração)

Tópico 1 · cada camada oferece uma interface mais ABSTRATA que a de baixo

Aprofundamento

A ideia de máquina de níveis aparece em Tanenbaum e em Machado & Maia: o computador é visto como uma pilha de camadas, e cada camada oferece à camada de cima uma máquina mais conveniente do que a que ela própria recebeu da camada de baixo. Um programa no topo nunca executa 'no hardware'; executa na máquina abstrata que a camada imediatamente inferior lhe apresenta.

No nível mais baixo estão os dispositivos físicos — transistores, portas lógicas, barramentos. Acima deles, a microarquitetura implementa as instruções: registradores, unidade lógico-aritmética, caminho de dados. O conjunto de instruções (ISA — Instruction Set Architecture) é a primeira camada que um programador consegue enxergar; é a fronteira entre hardware e software, e é nela que o fabricante do processador e o autor do sistema operacional se encontram.

O núcleo (kernel) do sistema operacional é a camada seguinte. Ele acrescenta abstrações que a ISA não tem (processos, arquivos, memória virtual) e é o único software que roda em modo kernel. Mas SO não é sinônimo de núcleo: boa parte do sistema operacional roda em modo usuário mesmo em sistemas monolíticos — daemons, serviços, gerenciador de janelas, o próprio init — e, em arquitetura de micronúcleo, até o sistema de arquivos, os drivers e a pilha de rede ficam em modo usuário de propósito. Quem internaliza 'SO = a caixa em modo kernel' erra qualquer questão sobre microkernel. Acima do núcleo, aplicações e utilitários (shell, compiladores) são programas comuns, lado a lado — o navegador não roda 'sobre' o compilador; a camada de utilitários existe em Tanenbaum como agrupamento funcional, não como pilha de dependência em execução.

Um alerta sobre a leitura do desenho: ele sugere que tudo passa pelo núcleo, e isso é falso. Um programa em modo usuário executa suas instruções diretamente na CPU — um laço somando inteiros não toca no SO em momento nenhum. O núcleo só entra em cena quando há uma operação que o programa não pode fazer sozinho: chamada de sistema, interrupção, exceção, falta de página. Ele não é uma camada interposta no caminho de execução; é uma camada que o programa invoca quando precisa. E o par modo usuário / modo kernel é um mecanismo de hardware — o bit de modo está na CPU e é definido pela ISA —, não uma propriedade das camadas de software.

Cuidado de nomenclatura: 'máquina de níveis' (esta hierarquia de abstração, de Tanenbaum) não é o mesmo que 'sistema operacional em camadas' (a arquitetura do THE, de Dijkstra, em que o núcleo é dividido internamente em níveis — assunto do aprofundamento sobre arquiteturas de núcleo). Estudantes confundem as duas com frequência.

A consequência prática da organização em níveis é a portabilidade: trocar o hardware exige reescrever apenas a parte do SO que conversa com ele (os drivers e o código dependente de arquitetura); as aplicações não mudam. Foi essa propriedade que permitiu ao Unix, e depois ao Linux, rodar em dezenas de arquiteturas diferentes com o mesmo conjunto de programas de usuário.

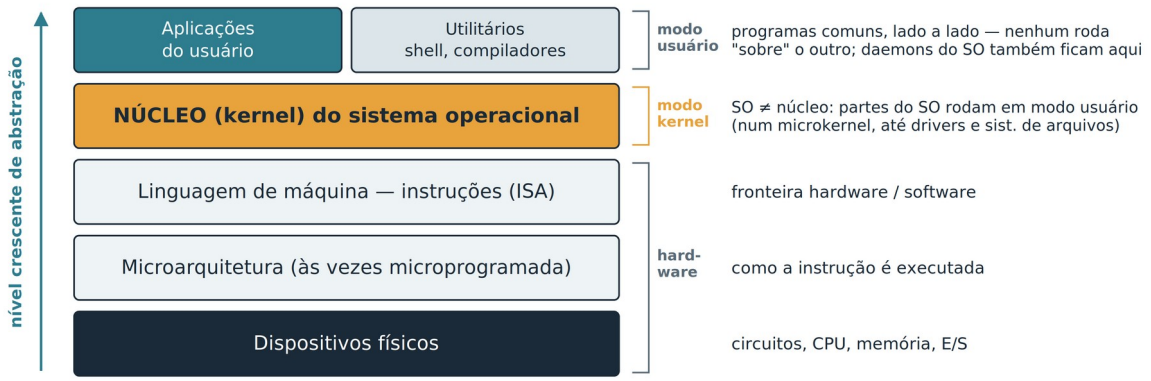


Figura 1 — O computador como máquina de níveis.

COMO LER A FIGURA

A figura deve ser lida de baixo para cima, acompanhando a seta da esquerda. Cada barra é uma camada; cada camada usa apenas os serviços da camada imediatamente abaixo. As anotações à direita dão um exemplo concreto do que vive em cada nível.

As chaves laterais mostram a divisão entre hardware (três camadas inferiores), modo kernel (só o núcleo) e modo usuário (aplicações e utilitários, desenhados lado a lado porque são programas de mesmo nível). As anotações à direita do núcleo lembram que SO ≠ núcleo. As três notas de rodapé são as três leituras erradas que o desenho costuma induzir, e vale enunciá-las em voz alta: o núcleo não fica no caminho das instruções comuns; o bit de modo é da CPU; e esta hierarquia não é o SO em camadas do THE.

Softwares tradutores: montador, compilador e interpretador

Tópico 1 · como o código que você escreve chega ao processador — tema apresentado em detalhe pela squad de Compiladores; aqui está o essencial para a questão 1 do caderno

Síntese

Montador (assembler) — traduz linguagem simbólica de montagem (mnemônicos) em código de máquina — em geral uma instrução para uma.

Compilador — traduz todo o código-fonte de alto nível para um módulo objeto ANTES da execução; erros de sintaxe aparecem em tempo de compilação; o executável roda sem o compilador.

Interpretador — traduz e executa comando a comando, sem gerar módulo objeto; mais lento e precisa estar presente a cada execução, mas é portável e facilita depurar (Python, shell).

Ligador (linker) e carregador (loader) — o ligador junta módulos objeto e bibliotecas resolvendo referências externas e produz o módulo de carga; o carregador o coloca na memória e o SO cria o processo.

Aprofundamento

A Matriz de Referência lista, no Tópico 1, as categorias de softwares tradutores, e a questão 1 do caderno as cobra em formato de lacunas. O que a prova espera é a distinção pelo MOMENTO da tradução. O montador faz uma tradução quase direta, de mnemônicos (MOV, ADD, JMP) para o código binário da ISA; o compilador lê o programa inteiro numa linguagem de alto nível, faz análise léxica, sintática e semântica, e gera um módulo objeto que só será executado depois — por isso os erros de sintaxe aparecem antes de qualquer execução e o programa final não precisa do compilador para rodar; o interpretador não gera módulo objeto: lê, traduz e executa um comando de cada vez, o que o torna mais lento e exige sua presença a cada execução, mas facilita a depuração e a portabilidade.

Entre o compilador e o processo há mais dois programas do sistema. O ligador (linker, ou link-editor) junta os módulos objeto do programa com as bibliotecas de que ele depende, resolvendo referências externas (a chamada a printf num módulo aponta para o código de printf em outro) e produzindo o módulo de carga, o executável. O carregador (loader) é a parte do SO que lê esse executável, coloca-o na memória, aplica a relocação se ela for estática e cria o processo. Bibliotecas dinâmicas adiam parte da ligação para o momento da carga ou da execução.

Há também o caso híbrido, que a prova usa como distrator: Java compila para bytecode, que é interpretado (ou compilado sob demanda, just-in-time) pela máquina virtual. Não é 'compilado' nem 'interpretado' no sentido puro — e a resposta correta costuma ser justamente essa.

Como os SO são classificados: três eixos

Tópico 1 · três famílias (Machado & Maia) — o mesmo sistema recebe uma coordenada em cada eixo

Síntese projetada

Eixo 1 — Programas e usuários — monoprogramação × multiprogramação; multitarefa cooperativa × preemptiva; monousuário × multiusuário.

Eixo 2 — Modo de processamento — em lote (batch) × tempo compartilhado (interativo) × tempo real (crítico/hard × brando/soft) — tempo compartilhado e tempo real pressupõem multiprogramação; o lote clássico da 2ª geração era monoprogramado.

Eixo 3 — Múltiplos processadores — um processador × multiprocessador; fortemente acoplados (memória compartilhada, um só SO: SMP × AMP) × fracamente acoplados (cada nó com seu SO: sistema operacional de rede × sistema distribuído).

Aprofundamento

As listas de 'tipos de SO' dos livros clássicos misturam critérios diferentes sob um rótulo só. Os três eixos usados aqui são as três famílias de Machado & Maia — exatamente o que a Matriz de Referência cita: programas e usuários (monoprogramáveis × multiprogramáveis, e quantas pessoas), modo de processamento (como o trabalho é submetido e atendido: em lote, em tempo compartilhado ou em tempo real) e múltiplos processadores (fortemente × fracamente acoplados). As classificações aparecem em prova quase sempre como pano de fundo de uma decisão de projeto: 'para um sistema de controle de tráfego aéreo, qual característica é indispensável?' A resposta depende de reconhecer que se trata de tempo real crítico, em que cumprir o prazo importa mais que a média de desempenho.

O primeiro eixo exige separar três termos que a linguagem corrente achata em um só — e essa confusão é matéria-prima de questão. MULTIPROGRAMAÇÃO é manter vários processos residentes na memória ao mesmo tempo para que a CPU nunca fique ociosa: quando um processo bloqueia esperando E/S, o SO entrega a CPU a outro. O objetivo é aproveitamento do processador, e não há promessa nenhuma de interatividade — um sistema multiprogramado pode deixar um processo longo ocupar a CPU por minutos, desde que ela esteja ocupada.

MULTITAREFA é a multiprogramação acrescida de preempção por fatia de tempo: o SO não espera o processo bloquear, ele o interrompe periodicamente e reveza a CPU em intervalos curtos, criando a ilusão de que tudo executa ao mesmo tempo. Todo sistema multitarefa é multiprogramado; a recíproca não vale. E aqui cabe um segundo corte, também cobrado: na multitarefa COOPERATIVA quem devolve a CPU é o próprio processo, que precisa ceder a vez voluntariamente — se ele travar ou entrar em laço infinito, o sistema inteiro congela; na PREEMPTIVA quem retoma a CPU é o escalonador, apoiado na interrupção do temporizador, e nenhum processo consegue monopolizar a máquina. É exatamente a diferença entre o Windows 3.x (cooperativo) e o Windows 95 (preemptivo), e entre o Mac OS 9 e o Mac OS X.

MULTIPROCESSAMENTO é outra coisa ainda: é execução de fato paralela, em mais de um núcleo ou processador físico. Multiprogramação e multitarefa são concorrência — os processos progridem alternadamente, e num único núcleo apenas um executa a cada instante. Multiprocessamento é paralelismo — dois processos executam

literalmente ao mesmo tempo, em unidades de processamento distintas. O subcorte usual é entre SMP (multiprocessamento simétrico), em que todos os núcleos são equivalentes, compartilham a memória e qualquer um pode executar código do núcleo, e AMP (assimétrico), em que há um processador mestre que executa o SO e distribui tarefas aos escravos. Praticamente todo sistema de propósito geral hoje é SMP. O multiprocessamento é o terceiro eixo, detalhado adiante; fecha o primeiro a distinção monousuário × multiusuário, que trata de quantas pessoas podem usar o sistema simultaneamente com contas, permissões e isolamento próprios.

O segundo eixo é o modo de processamento. Em lote (batch), trabalhos são submetidos previamente e executados sem interação; o objetivo é maximizar a vazão (throughput). Em tempo compartilhado (time-sharing), usuários interativos são atendidos com prioridade ao tempo de resposta. Em tempo real, o critério não é média nenhuma: é cumprir prazos — é a terceira modalidade de processamento de Machado & Maia, e é assim que a prova a classifica. Historicamente esses modos davam nome ao SO inteiro, porque o sistema era construído para um único deles: o OS/360 era um monitor de lote que não sabia fazer sessão interativa; o CTSS e o Multics foram projetados para tempo compartilhado. Hoje, num SO de propósito geral, lote e tempo compartilhado são modos de uso que coexistem no mesmo kernel — o mesmo Linux roda o terminal interativo e uma fila de jobs do Slurm a noite inteira. A nomenclatura clássica continua sendo a das bancas, então é preciso conhecê-la, entendendo a ressalva.

Uma observação importante: os eixos não são completamente independentes. O tempo compartilhado e o tempo real pressupõem multiprogramação (o lote da 2ª geração era monoprogramado, um job por vez; só o lote multiprogramado da 3ª geração mantém vários residentes) — não faz sentido falar em 'monotarefa com tempo compartilhado', porque compartilhar o tempo entre usuários exige vários processos residentes revezando a CPU. Ou seja, o eixo 2 depende do eixo 1. Quem tenta combinar livremente todas as posições dos três eixos esbarra em combinações que não fecham, e vale dizer isso em voz alta ao apresentar.

Ainda no segundo eixo, o tempo real exige o corte entre crítico e brando — e ele é propriedade do REQUISITO da aplicação, não do kernel. No tempo real CRÍTICO (hard), perder um prazo é falha do sistema, com consequência potencialmente catastrófica: freio ABS, marca-passo, controle de voo, acionamento de airbag. No tempo real BRANDO (soft), o atraso degrada a qualidade mas não invalida o resultado: um quadro perdido numa videoconferência ou um engasgo na reprodução de áudio incomodam, não matam. Sem esse corte, 'prazo' vira palavra vaga — e a distinção é frequente em prova.

Um RTOS (FreeRTOS, VxWorks, QNX, Zephyr) oferece garantias estruturais que um SO comum não dá: limite superior conhecido para a latência de interrupção, escalonador preemptivo por prioridade, herança de prioridade para evitar inversão de prioridade, ausência de caminhos de código com tempo não limitado. Não se transforma um SO comum em RTOS mandando um processo 'em tempo real'; é por isso que existe o PREEMPT_RT, um conjunto de mudanças no próprio kernel Linux, integrado à linha principal em 2024. O terceiro eixo é o dos sistemas com múltiplos processadores, e o corte de Machado & Maia é o grau de acoplamento. Nos fortemente acoplados, vários processadores compartilham a mesma memória e um único sistema operacional os administra — é o multiprocessamento propriamente dito, simétrico (SMP: todos equivalentes, qualquer um executa o núcleo) ou assimétrico (AMP: um mestre executa o SO e distribui trabalho aos demais). Nos fracamente acoplados, cada nó tem memória e sistema operacional próprios e a comunicação se dá por rede: no sistema operacional de rede, o usuário sabe em que máquina está e acessa as demais explicitamente (um servidor de arquivos, por exemplo); no sistema distribuído, o conjunto aparece ao usuário como uma máquina só, e é o SO que decide onde cada coisa executa. Tanenbaum também classifica os SO por área de aplicação — embarcado, móvel, de mainframe, de sensores — e o vocabulário é útil para ler enunciados, mas não é o corte que a matriz nomeia nem apareceu em prova.

A melhor forma de fixar que os critérios são distintos é classificar sistemas reais em todos os eixos de uma vez. O Linux de desktop é multitarefa preemptivo e multiusuário (eixo 1), de tempo compartilhado (eixo 2) e fortemente acoplado SMP (eixo 3), com núcleo monolítico modular. O FreeRTOS é multitarefa preemptivo e monousuário (eixo 1), de tempo real — crítico ou brando conforme o requisito da aplicação (eixo 2) — e de um processador nas configurações usuais (eixo 3), com núcleo mínimo; 'embarcado' descreve o ambiente em que

ele roda, não uma coordenada da classificação. Ver o mesmo sistema receber uma coordenada em cada eixo ensina mais do que qualquer afirmação de que os eixos são independentes.

Modo usuário × modo kernel

Tópico 2 · o mecanismo de proteção que sustenta tudo

Síntese projetada

A diferença em uma linha — no modo KERNEL a CPU executa qualquer instrução e acessa toda a memória e todos os dispositivos; no modo USUÁRIO ela só executa o subconjunto não privilegiado e só acessa a memória do próprio processo.

Bit de modo — um bit no registrador de status do processador diz em qual modo a CPU está executando.

Instruções privilegiadas — desabilitar interrupções, acessar registradores de controladores, alterar a tabela de páginas. Só executam em modo kernel.

Tentou em modo usuário? — o HARDWARE gera uma exceção e transfere o controle ao núcleo, que normalmente encerra o processo.

Por que importa — sem isso, qualquer programa poderia derrubar o sistema ou ler a memória dos outros.

Aprofundamento

O mecanismo de dois modos (dual-mode) é a base de toda a proteção que um sistema operacional oferece. Sem ele, qualquer programa poderia ler a memória dos outros, desligar o temporizador e monopolizar a CPU, ou escrever diretamente no disco e corromper o sistema de arquivos. O livro-texto aberto OSTEP (Arpaci-Dusseau) chama a técnica de 'execução direta limitada': o programa roda diretamente na CPU, à velocidade máxima, mas o hardware impõe limites ao que ele pode fazer.

A implementação é simples e depende do processador: um ou mais bits no registrador de estado indicam o modo corrente. Em modo kernel (também chamado supervisor, privilegiado ou anel 0 na nomenclatura x86), todas as instruções são permitidas e toda a memória é acessível. Em modo usuário (anel 3), um subconjunto de instruções — as privilegiadas — é proibido, e a MMU restringe os acessos ao espaço de endereçamento do próprio processo.

São privilegiadas as instruções que, executadas por um programa qualquer, comprometeriam o sistema inteiro: desabilitar ou habilitar interrupções, alterar o registrador que aponta a tabela de páginas, executar E/S diretamente nos registradores dos controladores, alterar o próprio bit de modo, parar o processador (HALT). Se um processo em modo usuário tenta executar uma delas, o hardware não a executa: gera uma exceção e transfere o controle ao núcleo, que normalmente encerra o processo com um erro do tipo 'operação ilegal' ou 'violação de acesso'.

A transição de modo usuário para modo kernel só acontece por três caminhos, todos controlados: uma interrupção de hardware, uma exceção causada pela instrução em curso ou uma chamada de sistema deliberada. A transição inversa acontece quando o núcleo executa a instrução de retorno de trap (IRET no x86), que restaura o estado do processo e rebaixa o privilégio. O processo nunca escolhe entrar em modo kernel 'por conta própria' — ele só pode pedir, e é o hardware que troca o modo ao entrar no tratador que o SO instalou.

Interrupção, exceção e chamada de sistema

Tópico 2 · três formas de entrar no núcleo — não confunda

Síntese projetada

Interrupção — ASSÍNCRONA. Vem de um dispositivo EXTERNO: disco terminou, temporizador expirou, tecla foi pressionada. Não tem relação com a instrução em curso.

Exceção (trap) — SÍNCRONA e involuntária. Causada pela própria instrução: divisão por zero, endereço inválido, instrução privilegiada em modo usuário, falta de página.

Chamada de sistema — SÍNCRONA e VOLUNTÁRIA. O processo pede um serviço ao núcleo executando uma instrução de trap.

API ≠ system call — a biblioteca (libc, Win32) roda em modo usuário. Há funções que não geram nenhuma syscall e outras que geram várias.

Aprofundamento

Esta distinção é cobrada em praticamente toda prova de SO, e o erro mais comum é tratar 'interrupção' como termo genérico para qualquer desvio no fluxo de execução. Os três mecanismos compartilham a mecânica — o processador salva o estado, consulta uma tabela de vetores, troca para modo kernel e desvia para um tratador — mas diferem em origem e intenção. Dominar o critério de classificação vale mais que memorizar exemplos.

A interrupção é assíncrona e externa: quem a provoca é um dispositivo que opera em paralelo com a CPU — o disco terminou uma transferência, chegou um pacote de rede, o temporizador expirou, uma tecla foi pressionada. Ela pode ocorrer entre quaisquer duas instruções e não tem relação nenhuma com a instrução que estava sendo executada. É a interrupção do temporizador, em particular, que devolve periodicamente o controle ao SO e torna a preempção possível: sem ela, um processo em laço infinito nunca sairia da CPU.

A exceção (também chamada trap ou falha, dependendo do autor e da arquitetura) é síncrona e interna: é a própria instrução em execução que a causa. Divisão por zero, acesso a endereço inválido, instrução privilegiada em modo usuário, opcode inexistente e falta de página são exceções. Se o programa for reexecutado com a mesma entrada, a exceção ocorre no mesmo ponto. Algumas exceções são fatais para o processo (divisão por zero sem tratamento); outras são parte normal da operação (a falta de página é exatamente como a memória virtual funciona).

A chamada de sistema é síncrona, interna e voluntária: o processo quer um serviço que só o núcleo pode prestar — ler um arquivo, criar um processo, alocar memória, enviar dados pela rede — e executa uma instrução de trap deliberadamente para pedi-lo. É o único dos três caminhos em que o programa escolhe entrar no núcleo. A API de programação (a biblioteca C, a Win32, a POSIX) embrulha essas chamadas em funções comuns; a função de biblioteca roda em modo usuário, prepara os argumentos e, em algum ponto, executa o trap.

Daí a pegadinha 'API é sinônimo de chamada de sistema', que é falsa em dois sentidos. Primeiro, há funções de biblioteca que não geram nenhuma chamada de sistema (strlen, sqrt, boa parte da manipulação de strings). Segundo, há funções que geram várias (um printf com buffer pode acumular saída e só chamar write ocasionalmente; um fopen pode gerar open e fstat). A biblioteca é uma camada de conveniência em modo usuário; a chamada de sistema é o mecanismo de travessia da fronteira.

O caminho completo de uma chamada de sistema

Tópico 2 · da aplicação ao núcleo e de volta

Aprofundamento

Vale acompanhar passo a passo o que acontece quando um programa executa algo tão trivial quanto escrever uma string em um arquivo, porque a prova costuma perguntar 'em qual etapa ocorre a troca de modo' ou 'qual componente executa em modo usuário'.

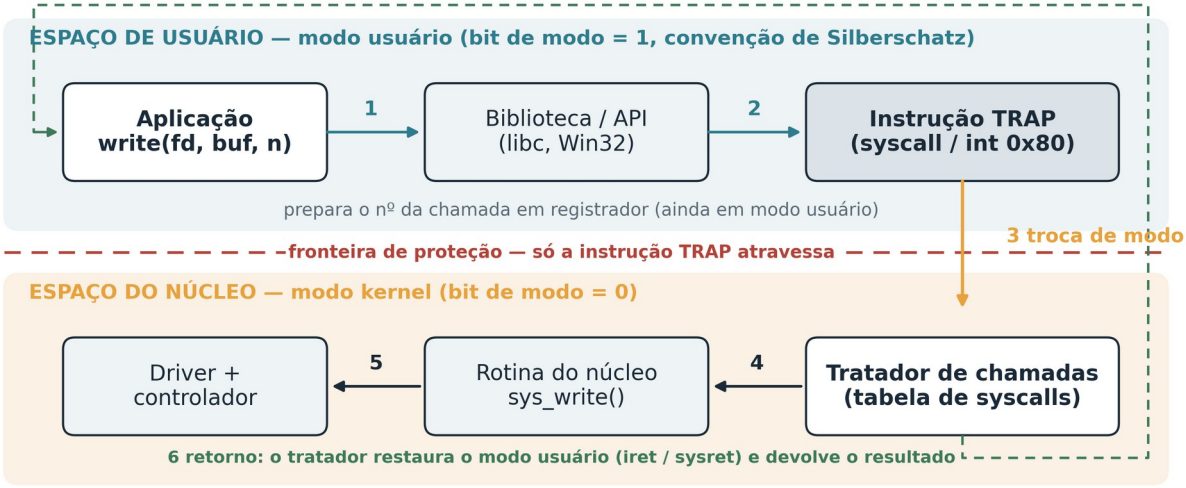
O programa chama write (ou fputs/printf, que passam antes por um buffer da biblioteca e só então chegam ao write — por isso uma função de biblioteca pode gerar zero, uma ou várias chamadas de sistema). A função de biblioteca roda em modo usuário; ela verifica argumentos, possivelmente copia os dados para um buffer interno e, quando decide que é hora de efetivar a operação, coloca o número da chamada de sistema (write tem um número fixo na tabela do núcleo) e os argumentos em registradores ou na pilha, segundo a convenção da arquitetura, e executa a instrução de trap — syscall no x86-64, int 0x80 no x86 antigo, svc no ARM.

A instrução de trap é o momento exato da travessia. O hardware salva o contador de programa e o registrador de estado, troca o bit de modo para kernel, troca para a pilha do núcleo (cada processo tem uma pilha de núcleo separada, para que o kernel nunca dependa da pilha do usuário, que pode estar corrompida) e desvia para o

endereço do tratador de chamadas de sistema — endereço que o núcleo registrou em uma tabela durante a inicialização e que o programa de usuário não conhece nem pode alterar.

O tratador de chamadas (o dispatcher de syscalls) lê o número da chamada, valida-o e o usa como índice em uma tabela de ponteiros para funções, chegando à rotina que implementa write. Essa rotina verifica se o descritor de arquivo é válido e pertence ao processo, se o processo tem permissão de escrita, se o buffer de origem está em memória acessível ao processo — validações que só fazem sentido em modo kernel, porque o processo poderia ter passado argumentos maliciosos — e então repassa o pedido ao sistema de arquivos e ao driver do dispositivo.

Concluída a operação (ou enfileirada, se for assíncrona), o núcleo coloca o valor de retorno em um registrador e executa a instrução de retorno de trap, que restaura o estado salvo, rebaixa o modo para usuário e retoma o programa na instrução seguinte ao trap. A função de biblioteca recebe o valor, converte um eventual código de erro em errno e retorna ao programa. Para o programa, tudo isso parece uma chamada de função comum — e é essa ilusão que a máquina estendida vende.



printf() e fputs() têm buffer: podem gerar 0, 1 ou várias chamadas write() — a função de biblioteca NÃO é a syscall

Figura 2 — Percurso completo de uma chamada de sistema.

COMO LER A FIGURA

A figura separa o espaço de usuário (acima) do espaço do núcleo (abaixo) pela linha tracejada vermelha, que representa a fronteira de proteção. Os passos numerados seguem a ordem temporal: 1 e 2 acontecem inteiramente em modo usuário; 3 é a única travessia descendente, feita pela instrução de trap; 4 e 5 acontecem em modo kernel; 6 é a travessia ascendente, feita pela instrução de retorno.

Observe que a biblioteca (libc, Win32) está do lado do usuário. É o ponto que a questão de V/F explora: a API não é a chamada de sistema, é quem a prepara. E observe que o tratador consulta uma tabela para localizar a rotina — o processo informa um número, nunca um endereço, e por isso não consegue desviar o núcleo para código arbitrário.

Arquiteturas de núcleo

Tópico 3 · onde cada serviço do SO é executado

Aprofundamento

A arquitetura do núcleo responde a uma pergunta de engenharia: onde cada serviço do SO deve executar, e com que privilégio? As respostas históricas formam um espectro que vai do núcleo monolítico — tudo em modo kernel, um único espaço de endereçamento — ao micronúcleo — o mínimo em modo kernel, o resto como servidores em modo usuário. A prova costuma pedir que o estudante associe cada arquitetura a suas vantagens e custos.

No núcleo monolítico, escalonador, gerência de memória, sistema de arquivos, pilha de rede e drivers formam um único programa executando em modo kernel. Qualquer parte pode chamar qualquer outra por chamada de função direta, o que dá o melhor desempenho. O custo é a confiabilidade: um erro em um driver de placa de som pode derrubar o sistema inteiro, porque está no mesmo espaço de endereçamento do escalonador. Linux, os BSDs e o núcleo original do Unix são monolíticos — embora o Linux seja modular, isto é, permita carregar e descarregar partes (módulos) em tempo de execução, o que dá flexibilidade sem mudar a arquitetura: o módulo carregado passa a executar em modo kernel como o resto.

A arquitetura em camadas, cuja referência histórica é o sistema THE de Dijkstra (1968), impõe uma disciplina ao núcleo monolítico: o software é dividido em níveis, e cada nível só pode usar serviços do nível imediatamente inferior. No THE, o nível 0 alocava o processador (multiprogramação básica), o nível 1 gerenciava memória e tambor, o nível 2 cuidava da comunicação com o console do operador, o nível 3 fazia a gerência de E/S e o nível 4 continha os programas do usuário — o operador humano era considerado o nível 5. A vantagem é de projeto e verificação: cada camada pode ser testada assumindo que as inferiores estão corretas. A desvantagem é que um pedido pode ter de atravessar várias camadas, cada uma acrescentando processamento, e que nem sempre é óbvio em qual camada um serviço deve ficar.

No micronúcleo, o kernel propriamente dito fica reduzido ao essencial — comunicação entre processos (IPC), escalonamento básico, gerência mínima de memória e tratamento de interrupções. Sistema de arquivos, drivers, pilha de rede e gerência de memória de alto nível viram servidores, processos comuns em modo usuário. Se o servidor de arquivos falhar, o núcleo continua de pé e pode reiniciá-lo. O custo é a comunicação: um pedido que no monolítico era uma chamada de função vira uma troca de mensagens entre processos, com trocas de modo e de contexto. MINIX 3, QNX (usado em sistemas automotivos) e o L4 são micronúcleos; o Mach foi a base do XNU, o núcleo híbrido do macOS e do iOS.

Os sistemas comerciais tendem a posições intermediárias, os chamados núcleos híbridos: o Windows NT e o XNU têm estrutura inspirada em micronúcleo, mas executam os serviços críticos em modo kernel por desempenho. Quando a prova pede o critério de comparação, os eixos são desempenho, confiabilidade (isolamento de falhas), segurança (superfície de código privilegiado) e extensibilidade (facilidade de acrescentar serviços sem recompilar o núcleo).

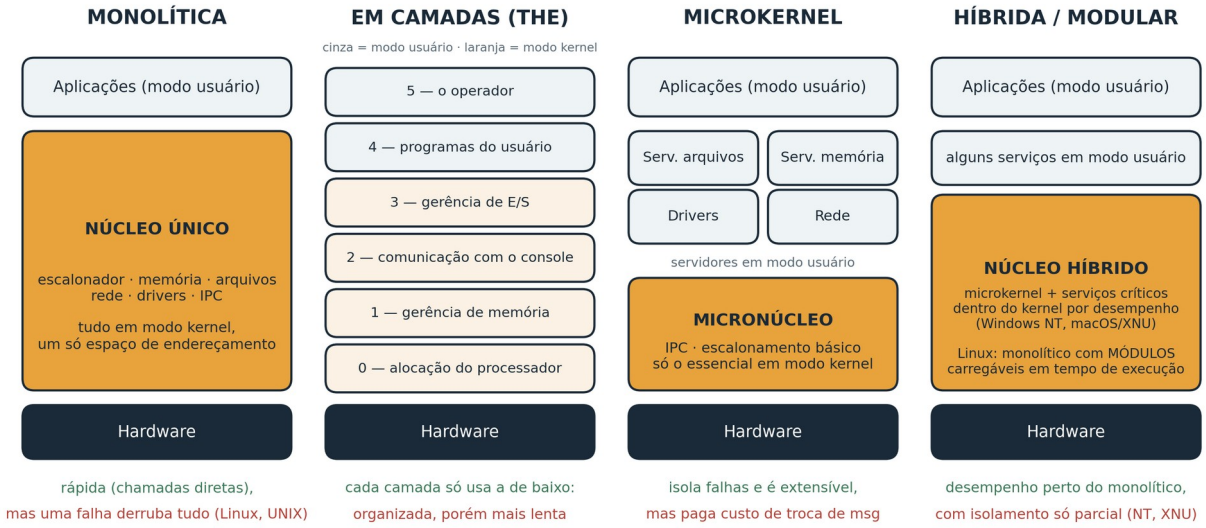


Figura 3 — Arquiteturas monolítica, em camadas, microkernel e híbrida/modular.

COMO LER A FIGURA

As quatro colunas mostram a mesma máquina organizada de quatro formas; na coluna em camadas, o tom laranja marca as camadas que rodam em modo kernel (0 a 3) e o cinza as de modo usuário (4 e 5); a quarta coluna, híbrida/modular, é o compromisso que Windows NT, macOS e Linux adotam. Na monolítica, a grande caixa laranja é o núcleo único em modo kernel; tudo que o SO faz está ali. Na arquitetura em camadas, as seis camadas do THE aparecem na ordem original de Dijkstra, do processador (camada 0) até o operador (camada 5); a regra é que cada uma só invoca a imediatamente inferior. No micronúcleo, a caixa laranja encolheu para 'IPC e escalonamento básico', e os serviços subiram para o espaço de usuário como servidores.

As frases em verde e vermelho abaixo de cada coluna resumem o trade-off que a questão vai pedir: desempenho versus isolamento de falhas. Não existe arquitetura 'melhor' — existe a adequada ao requisito.

Virtualização e contêineres

Tópico 3 · três formas de rodar mais de um ambiente na mesma máquina

Síntese projetada

Diferença essencial — hipervisor virtualiza HARDWARE — cada VM tem seu próprio núcleo. Contêiner compartilha UM núcleo.

Aprofundamento

Virtualizar é criar a ilusão de várias máquinas onde há uma só. O software que faz isso é o hipervisor (ou monitor de máquina virtual, VMM), e o critério de classificação que a prova usa é onde ele roda: diretamente sobre o hardware (tipo 1, bare metal) ou como uma aplicação sobre um sistema operacional hospedeiro (tipo 2, hosted).

O hipervisor tipo 1 — VMware ESXi, Microsoft Hyper-V, Xen, KVM (que transforma o próprio kernel Linux em hipervisor) — é o próprio 'sistema operacional' da máquina física. Cada máquina virtual tem seu SO convidado completo, com seu próprio kernel, que acredita estar controlando o hardware; o hipervisor intercepta as instruções privilegiadas e as emula ou as repassa, e as extensões de virtualização dos processadores (Intel VT-x, AMD-V) tornam essa interceptação eficiente. É a base de todos os provedores de nuvem: a 'instância' que se aluga é uma VM sobre um hipervisor tipo 1.

O hipervisor tipo 2 — VirtualBox, VMware Workstation, Parallels — é um programa comum instalado em um SO hospedeiro (Windows, macOS, Linux), e é sobre ele que as VMs rodam. É mais simples de instalar e adequado a desktop, laboratório e testes, mas paga o custo de uma camada extra: os pedidos da VM atravessam o hipervisor e depois o SO hospedeiro. Na prática a fronteira entre os dois tipos ficou menos nítida — o KVM roda dentro do Linux mas tem desempenho de tipo 1 — mas a distinção conceitual continua sendo a cobrada.

Os contêineres (Docker, Podman, LXC) resolvem um problema diferente e é essencial não confundi-los com VMs. Um contêiner não virtualiza hardware e não tem kernel próprio: todos os contêineres de uma máquina compartilham o mesmo núcleo do hospedeiro. O que o contêiner isola são os processos — cada um enxerga sua própria árvore de processos, sua rede, seu sistema de arquivos, seus usuários. Isso é feito com dois mecanismos do kernel Linux: namespaces (que dão a cada contêiner uma visão separada dos recursos) e cgroups (que limitam quanto de CPU, memória e E/S cada um pode consumir).

A consequência do compartilhamento do kernel é dupla. De um lado, contêineres são muito mais leves que VMs — iniciam em milissegundos, ocupam megabytes em vez de gigabytes, e uma máquina que suporta dez VMs suporta centenas de contêineres. De outro, o isolamento é menor: uma vulnerabilidade no kernel compartilhado expõe todos os contêineres, e um contêiner Linux não pode rodar um sistema Windows, porque não há kernel Windows ali. Por isso os provedores de nuvem executam contêineres de clientes distintos dentro de VMs distintas, combinando os dois mecanismos.

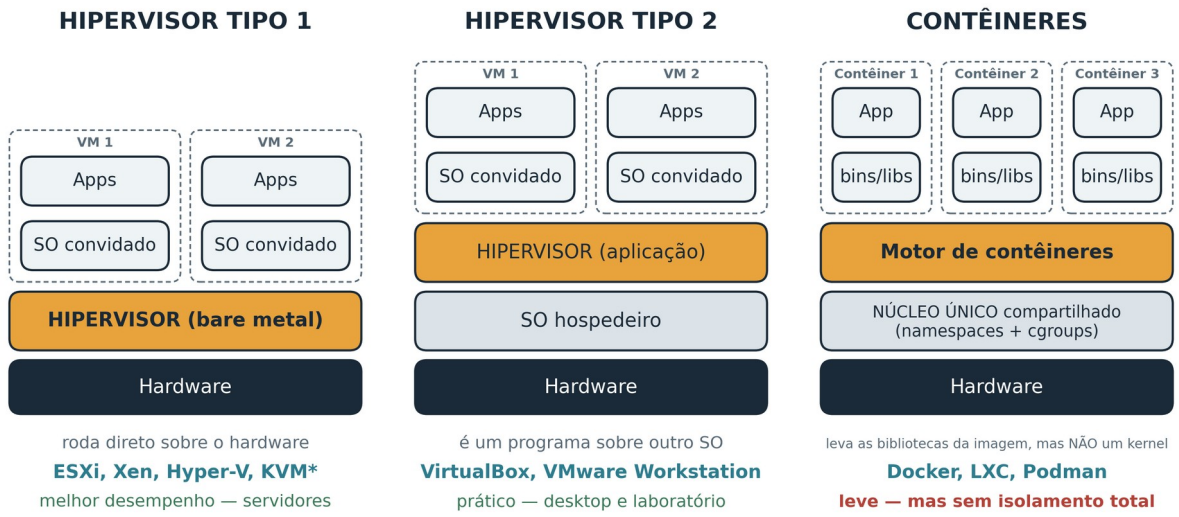


Figura 5 — Hipervisor tipo 1, tipo 2 e contêineres.

COMO LER A FIGURA

As três pilhas devem ser comparadas pelo que está imediatamente acima do hardware. No tipo 1 é o hipervisor; no tipo 2 é um sistema operacional hospedeiro completo, e o hipervisor é apenas uma aplicação sobre ele; nos contêineres é um único kernel, compartilhado por todos.

Conte os núcleos em cada pilha: nas duas primeiras, cada 'SO convidado' traz o seu, e por isso as VMs são pesadas e isoladas. Na terceira, há um só, e por isso os contêineres são leves e menos isolados. Essa contagem responde à maioria das questões sobre o tema.

Termos que a prova usa

VOCABULÁRIO DA SEÇÃO 1

Multiprogramação — vários processos residentes na memória para manter a CPU ocupada durante as operações de E/S.

Multitarefa — multiprogramação com preempção por fatia de tempo; cooperativa (o processo cede a vez) × preemptiva (o escalonador retoma).

Multiprocessamento — paralelismo real em mais de um núcleo; fortemente acoplados (memória compartilhada, um só SO: SMP × AMP) × fracamente acoplados (nós em rede: SO de rede × distribuído).

Tempo real crítico × brando — perder o prazo é falha do sistema × degrada a qualidade sem invalidar o resultado.

Máquina estendida — visão do SO como fornecedor de abstrações que escondem o hardware.

Gerenciador de recursos — visão do SO como árbitro de CPU, memória e dispositivos entre programas concorrentes.

ISA — conjunto de instruções do processador; fronteira entre hardware e software.

Modo kernel / modo usuário — os dois níveis de privilégio do processador; só o primeiro executa instruções privilegiadas.

Instrução privilegiada — instrução que só executa em modo kernel; em modo usuário gera exceção.

Trap — instrução (ou evento) que transfere o controle ao núcleo trocando o modo.

Interrupção — evento assíncrono gerado por hardware externo à CPU.

Exceção — evento síncrono causado pela instrução em execução.

Chamada de sistema — pedido voluntário de serviço ao núcleo, via trap.

API — interface de programação em modo usuário que embrulha chamadas de sistema.

Núcleo monolítico / micronúcleo — todo o SO em modo kernel × só o essencial em modo kernel.

Hipervisor tipo 1 / tipo 2 — sobre o hardware × sobre um SO hospedeiro.

Contêiner — isolamento de processos sobre um kernel compartilhado (namespaces + cgroups).

POST, MBR, GPT, initramfs, PID 1 — marcos da sequência de boot.

Erros mais comuns

ONDE OS ESTUDANTES ERRAM

- Usar multiprogramação, multitarefa e multiprocessamento como sinônimos. Só o terceiro envolve paralelismo real; multitarefa é multiprogramação com preempção por quantum.
- Combinar monotarefa com tempo compartilhado. Tempo compartilhado e tempo real pressupõem multiprogramação; o lote monoprogramado da 2ª geração é a exceção histórica.
- Tratar tempo real como sinônimo de rápido. É previsibilidade e prazo, não velocidade; e crítico (hard) não é o mesmo que brando (soft).
- Chamar de 'interrupção' qualquer evento que desvie o fluxo — divisão por zero e falta de página são exceções, não interrupções.
- Achar que a função da biblioteca C é a chamada de sistema. A biblioteca roda em modo usuário; a chamada é o trap que ela executa.
- Colocar o GRUB antes do firmware ou o init antes da montagem da raiz na sequência de boot.
- Tratar contêiner como 'VM leve'. Contêiner não tem kernel próprio.
- Confundir modular com micronúcleo. O Linux é modular e monolítico: o módulo carregado executa em modo kernel.

Questões desta seção

Consulte o caderno de questões: questões 1, 2 e 3. O gabarito comentado, com a análise de cada distrator, está no documento de gabarito.

Seção 2 — Processos, Sincronização e Escalonamento

Tópicos 4 e 5 — PCB, estados, concorrência, deadlock e uso da CPU

- T4 · Processos e threads: PCB, estados, concorrência, sincronização e deadlock
- T5 · Compartilhamento do processador e algoritmos de escalonamento

Esta é a seção mais densa da revisão e a que concentra o maior número de questões históricas do ENADE em Sistemas Operacionais: threads caíram em 2008, 2011 e 2014; sincronização com semáforos (produtor-consumidor) em 2005 e 2014; escalonamento (algoritmo × ambiente) e região crítica com desabilitação de interrupções em 2021; deadlock com semáforos em 2017. Ela corresponde ao Objetivo de Aprendizagem 2 da Matriz de Referência, que junta processos, threads, concorrência e escalonamento sob um único guarda-chuva — e por boa razão: são as várias faces de um mesmo problema, o de fazer muitas coisas acontecerem sobre poucas CPUs sem que elas se atrapalhem.

A ordem escolhida para a fala é: o que é um processo e como o SO o representa (PCB); os estados pelos quais ele passa; o que muda quando o processo vira multithread, e quem escalona a thread (biblioteca ou núcleo); o que acontece quando processos ou threads precisam coordenar o acesso a algo compartilhado — condições de corrida, exclusão mútua, semáforos, produtor-consumidor e o impasse, com suas estratégias de tratamento; e, por fim, como o escalonador decide quem usa a CPU e qual algoritmo serve a qual ambiente. A questão da seção fecha o bloco.

Processo e o Bloco de Controle de Processo

Tópico 4 · a estrutura que representa o processo dentro do núcleo

Síntese projetada

Contexto de hardware — registradores, contador de programa (PC), ponteiro de pilha e PSW — o que precisa ser salvo e restaurado.

Troca de contexto é overhead — nenhum trabalho útil do usuário é feito durante ela.

Aprofundamento

Um programa é um arquivo passivo no disco; um processo é esse programa em execução — com um contador de programa indicando a próxima instrução, registradores com valores, uma pilha, uma área de dados, arquivos abertos. Dois processos podem executar o mesmo programa (duas janelas do mesmo editor) e são inteiramente independentes. Essa distinção programa/processo abre muitas questões e vale ser dita com clareza.

Para gerenciar processos o núcleo precisa representá-los em uma estrutura de dados, e essa estrutura é o Bloco de Controle de Processo (PCB) — task_struct no Linux, EPROCESS no Windows. Existe um PCB por processo, e o conjunto deles forma a tabela de processos. Machado & Maia organizam o conteúdo do PCB em três partes que a prova costuma nomear: contexto de hardware, contexto de software e espaço de endereçamento.

O contexto de hardware é o conteúdo dos registradores do processador no instante em que o processo perdeu a CPU: contador de programa, ponteiro de pilha, registradores de propósito geral, registrador de estado (PSW). É o que precisa ser salvo para que o processo possa retomar exatamente de onde parou — a questão sobre o núcleo didático cujos processos 'reiniciam do começo' descreve justamente um sistema que não salva esse contexto.

O contexto de software reúne o que caracteriza o processo perante o SO: identificação (PID, PID do pai, usuário e grupo proprietários), prioridade e política de escalonamento, quotas e limites (máximo de arquivos abertos, de memória, de tempo de CPU), estatísticas de contabilização, sinais pendentes, e o estado atual (pronto, executando, esperando). O espaço de endereçamento é a memória que o processo pode acessar — descrita, na prática, por um ponteiro para a tabela de páginas do processo.

A troca de contexto é a operação em que o núcleo retira um processo da CPU e coloca outro: salva o contexto de hardware do primeiro em seu PCB, escolhe o próximo (isso é o escalonador), restaura o contexto de hardware do segundo a partir de seu PCB e aponta a MMU para a tabela de páginas do novo processo. É uma operação puramente de overhead — nenhuma instrução útil de usuário executa durante ela — e leva da ordem de microssegundos, o que explica por que um quantum de escalonamento muito curto degrada o desempenho: a fração do tempo gasta trocando de contexto cresce.

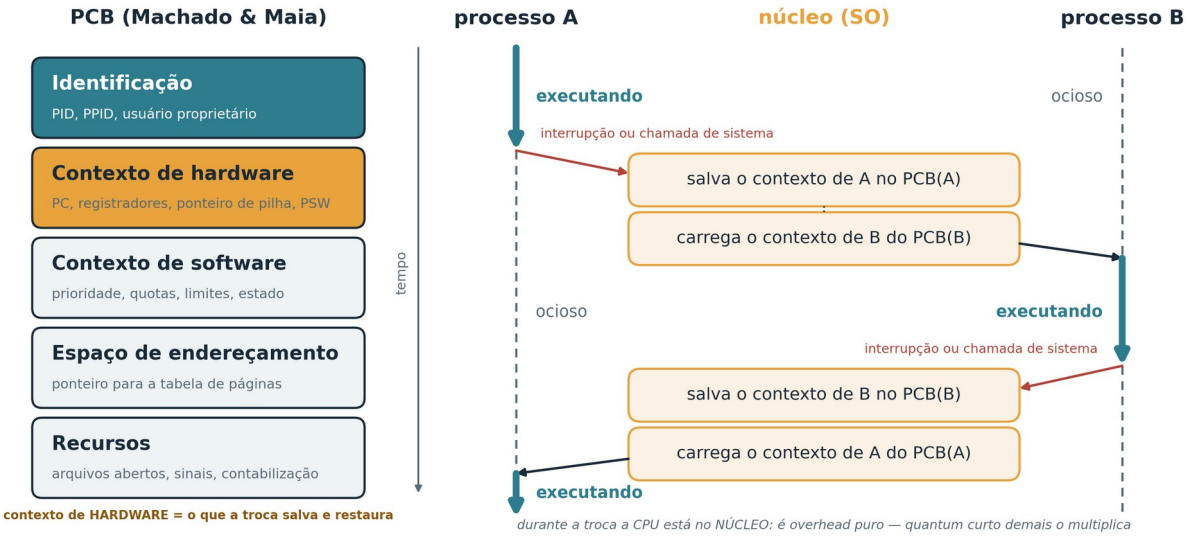


Figura 7 — Conteúdo do PCB e mecânica da troca de contexto.

COMO LER A FIGURA

À esquerda, o PCB dividido em blocos: identificação, contexto de hardware (destacado em laranja porque é o que a troca de contexto salva e restaura), contexto de software, espaço de endereçamento e recursos. À direita, a linha do tempo de uma troca de contexto, no formato de Silberschatz (Fig. 3.6): o tempo desce, a seta cheia marca o processo que executa e o tracejado o que está ocioso; a coluna central é o núcleo. A executa; ocorre uma interrupção ou chamada de sistema; o núcleo salva o contexto de A no PCB(A), carrega o de B do PCB(B); B executa.

O intervalo entre 'salva' e 'restaura' é o custo da troca — tempo em que a CPU está ocupada com o núcleo e nenhum dos dois processos avança. É por isso que o diagrama termina com a frase 'é puro overhead'.

Estados e transições do processo

Tópico 4 · o ciclo de vida que o escalonador controla

Síntese projetada

A pegadinha clássica — não existe transição ESPERA → EXECUÇÃO. Terminada a E/S, o processo volta para PRONTO e disputa a CPU de novo.

Aprofundamento

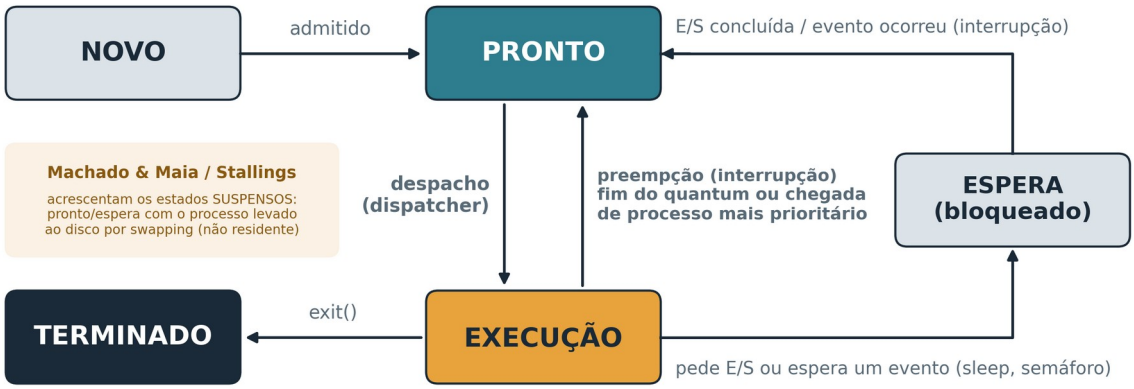
Ao longo da vida, um processo alterna entre estados que descrevem o que ele está fazendo do ponto de vista do SO. O modelo mínimo tem três: executando (de posse da CPU), pronto (poderia executar, mas a CPU está com outro) e esperando ou bloqueado (não pode executar porque aguarda algo — a conclusão de uma E/S, a chegada de um sinal, a liberação de um semáforo). Silberschatz acrescenta novo (em criação) e terminado (encerrado, aguardando que o pai colete seu status), chegando aos cinco estados do diagrama.

As transições têm nomes que a prova usa. Novo → pronto é a admissão. Pronto → executando é o despacho, feito pelo dispatcher após decisão do escalonador. Executando → pronto é a preempção — o processo não

terminou, mas perdeu a CPU porque o quantum expirou ou porque chegou um processo mais prioritário. Executando → esperando ocorre quando o processo pede algo que não está imediatamente disponível (uma leitura de disco, um recurso ocupado). Esperando → pronto ocorre quando o que ele aguardava acontece, normalmente sinalizado por uma interrupção. Executando → terminado é a saída (exit).

A transição que não existe é a que mais cai em prova: esperando → executando. Quando a E/S termina, o processo não volta direto para a CPU; volta para a fila de prontos e disputa a CPU como qualquer outro. Pode até ser que o escalonador o escolha imediatamente, mas ele passou por pronto. Da mesma forma, um processo em espera não pode ser preemptado, porque não está de posse de nada.

Em um sistema monoprocessado, no máximo um processo está em executando a cada instante; todos os demais estão em pronto ou esperando. A fila de prontos é a estrutura sobre a qual o escalonador trabalha, e há uma fila de espera por dispositivo ou por evento. Sistemas reais têm estados adicionais — suspenso em disco (swapped out), zumbi, parado — mas o modelo de cinco estados é o que a prova adota.



só a transição PRONTO → EXECUÇÃO é decisão do escalonador; as demais são consequência de eventos

Figura 6 — Estados do processo e transições possíveis.

COMO LER A FIGURA

Os cinco retângulos são os estados; as setas são as transições, rotuladas com o evento que as provoca. Repare no par de setas verticais entre PRONTO e EXECUÇÃO: descem pelo despacho e sobem pela preempção — é o ciclo de vida normal de um processo interativo, que ganha e perde a CPU dezenas de vezes por segundo.

Repare também no caminho ESPERA → PRONTO → EXECUÇÃO no lado direito. Não há seta de ESPERA para EXECUÇÃO; a frase do rodapé lembra que só PRONTO → EXECUÇÃO é decisão do escalonador — as demais transições são consequência de eventos. A caixa à esquerda registra os estados suspensos de Machado & Maia e Stallings: pronto/espera com o processo levado ao disco por swapping. Só EXECUÇÃO é destacado em laranja porque é o único estado que ocupa a CPU.

Processo × thread

Tópico 4 · o que é compartilhado e o que é privado

Síntese projetada

Regra de ouro — threads do mesmo processo compartilham código, dados e arquivos; cada uma tem apenas PC, registradores e pilha próprios.

Aprofundamento

Um processo tradicional tem um único fluxo de execução: um contador de programa, uma pilha. A thread (ou linha de execução, ou processo leve) é a separação dessas duas ideias: o processo continua sendo a unidade de posse de recursos — espaço de endereçamento, arquivos abertos, sinais — mas passa a poder conter vários fluxos de execução, cada um com seu próprio contador de programa, seus registradores e sua pilha.

O que as threads de um mesmo processo compartilham é tudo o resto: o código, as variáveis globais e o heap, os descritores de arquivo, o diretório corrente. Isso torna a comunicação entre elas trivial — basta escrever em uma variável que a outra lê — e torna a criação e a troca de contexto entre threads muito mais baratas que entre processos, porque não há espaço de endereçamento a trocar nem tabela de páginas a recarregar (e portanto a TLB não é invalidada).

O preço da facilidade é a ausência de proteção. Duas threads escrevendo na mesma variável sem coordenação produzem uma condição de corrida; uma thread que corrompe o heap corrompe-o para todas; uma thread que faz um acesso inválido derruba o processo inteiro. Processos separados não têm esses problemas — mas comunicar-se entre eles exige mecanismos de IPC (pipes, sockets, memória compartilhada explícita) mediados pelo núcleo. A escolha entre 'vários processos' e 'várias threads' é um trade-off entre isolamento e eficiência, e a prova gosta de pedi-lo em cenários (um navegador, um servidor web, um jogo).

Há ainda a distinção entre threads de usuário e threads de núcleo. Threads de usuário são gerenciadas por uma biblioteca em modo usuário, sem que o núcleo saiba de sua existência: são baratíssimas, mas se uma delas bloqueia em uma chamada de sistema, o núcleo bloqueia o processo inteiro, e não há paralelismo real em múltiplas CPUs. Threads de núcleo são conhecidas e escalonadas pelo SO — o que dá paralelismo e bloqueio independente, ao custo de trocas de contexto que passam pelo núcleo. O modelo um-para-um (cada thread de usuário sobre uma thread de núcleo) é o adotado por Linux e Windows.

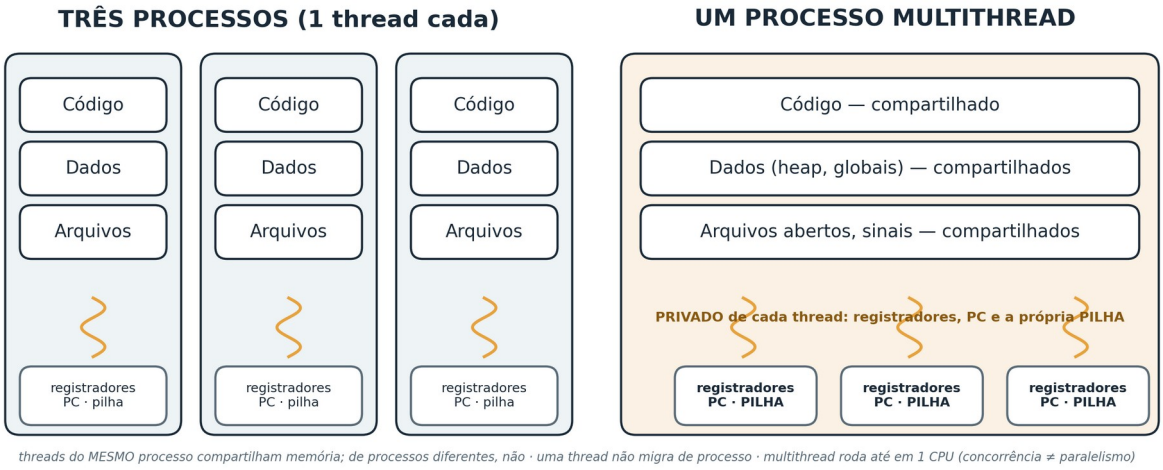


Figura 8 — O que é compartilhado entre threads do mesmo processo.

COMO LER A FIGURA

À esquerda, três processos independentes: cada caixa externa é um espaço de endereçamento separado, com seu código, dados e arquivos, e uma única thread (a linha ondulada) dentro. À direita, um único processo cujo espaço de endereçamento contém código, dados e arquivos compartilhados e três threads, cada uma com seus próprios registradores, contador de programa e PILHA — a pilha própria é o item mais cobrado.

A pergunta que a figura responde é 'o que é privado e o que é compartilhado'. Privado de cada thread: registradores, contador de programa, pilha. Compartilhado por todas: tudo o mais. A linha do rodapé reúne as três afirmações que a prova já usou: threads de processos diferentes não compartilham memória; uma thread não migra de processo; multithread roda mesmo em um único núcleo (concorrência), e só ganha paralelismo com vários núcleos.

Threads de usuário × threads de kernel

Tópico 4 · quem escalona a thread: a biblioteca ou o núcleo?

Síntese projetada

Threads de usuário (modo usuário) — criadas e escalonadas por uma biblioteca; o kernel enxerga UM só fluxo. Troca de contexto barata, mas se uma thread bloqueia em E/S o processo inteiro bloqueia, e não há paralelismo real em multicore.

Threads de kernel — o núcleo conhece e escalona cada thread: uma bloqueada não trava as outras e várias rodam em paralelo nos núcleos. Troca mais cara (passa pelo kernel). É o modelo do Linux e do Windows.

Modelos de mapeamento — muitos-para-um (só usuário) · um-para-um (Linux, Windows) · muitos-para-muitos (M:N, Solaris antigo).

O que a prova pergunta (2008, 2011, 2014) — threads do mesmo processo compartilham código, dados e arquivos; cada uma tem PC, registradores e PILHA próprios; não migram entre processos; multithread roda em 1 núcleo (concorrência) e só ganha paralelismo com vários núcleos.

Aprofundamento

O que muda entre os dois tipos é quem sabe da existência da thread. Nas threads de usuário, uma biblioteca em modo usuário cria, escalona e troca as threads sem nenhuma chamada de sistema; para o núcleo existe um único processo com um único fluxo. É barato — a troca não passa pelo kernel — mas tem dois defeitos que a prova explora: se uma thread faz uma chamada de sistema bloqueante, o núcleo bloqueia o processo inteiro, e em uma máquina multicore nunca há duas threads do mesmo processo executando ao mesmo tempo, porque o núcleo só enxerga uma. Nas threads de kernel, cada thread é uma entidade escalonável pelo SO: bloqueia sozinha, roda em paralelo nos núcleos disponíveis, e paga por isso uma troca de contexto que passa pelo kernel. Os modelos de mapeamento descrevem a relação entre threads de usuário e de kernel: muitos-para-um (só biblioteca), um-para-um (cada thread de usuário sobre uma de kernel — Linux e Windows) e muitos-para-muitos (um conjunto de threads de usuário multiplexado sobre um conjunto menor de threads de kernel, como no Solaris antigo).

Critérios: o que se quer otimizar

Tópico 5 · não existe algoritmo bom para tudo

Síntese projetada

Utilização de CPU — manter o processador ocupado — maximizar.

Throughput (vazão) — processos concluídos por unidade de tempo — maximizar.

Turnaround — conclusão menos chegada — o tempo total do ponto de vista do usuário — minimizar.

Tempo de espera — quanto tempo o processo passou na fila de PRONTO — minimizar.

Tempo de resposta — da submissão até a PRIMEIRA resposta. É o critério dos sistemas interativos — minimizar.

Aprofundamento

Antes de comparar algoritmos é preciso saber o que se está medindo, porque nenhum algoritmo é bom em todos os critérios ao mesmo tempo — e é essa tensão que as questões exploram. Silberschatz lista cinco critérios, e vale saber a definição exata de cada um porque as questões de cálculo pedem um deles pelo nome.

Utilização da CPU é a fração do tempo em que o processador está executando algo útil; quer-se maximizá-la. Throughput (vazão) é o número de processos concluídos por unidade de tempo; também se maximiza. Os três critérios restantes são tempos, e quer-se minimizá-los: turnaround (tempo de retorno) é o intervalo entre a submissão do processo e sua conclusão — inclui todo o tempo de espera, de execução e de E/S; tempo de espera

é o total de tempo que o processo passou na fila de prontos, sem contar execução nem E/S; tempo de resposta é o intervalo entre a submissão e a primeira resposta produzida, não a conclusão.

Para fins de prova, com processos que não fazem E/S: turnaround = instante de conclusão − instante de chegada; espera = turnaround − duração (burst). O tempo de resposta, em sistemas interativos, é o que importa ao usuário — ele não se incomoda que uma compilação demore, mas se incomoda se o editor demorar a ecoar uma tecla. Por isso sistemas de tempo compartilhado otimizam resposta, sistemas em lote otimizam vazão e turnaround, e sistemas de tempo real não otimizam média nenhuma: garantem prazos.

Uma sutileza que cai em prova: minimizar a média de um critério não é o mesmo que minimizar sua variância. Um sistema interativo prefere tempos de resposta previsíveis a tempos médios ligeiramente menores mas ocasionalmente muito longos. O Round-Robin é valorizado exatamente por isso.

Os algoritmos de escalonamento

Tópico 5 · qual algoritmo resolve qual problema

Síntese projetada

FIFO / FCFS — não preemptivo. Simples e justo na ordem, mas um processo longo atrasa todos (efeito comboio). Sistemas em lote.

SJF e SRTF — menor tarefa primeiro; SRTF é a versão preemptiva. Dá o menor tempo médio de espera — para chegadas simultâneas e sem preempção —, mas exige prever a duração e causa inanição dos processos longos.

Round-Robin — preemptivo por quantum. Limita o tempo de resposta — é a escolha para sistemas de tempo compartilhado. Quantum pequeno demais = excesso de troca de contexto; grande demais = vira FCFS.

Por prioridade e múltiplas filas — atende classes distintas; prioridade fixa causa inanição → envelhecimento (aging). Múltiplas filas COM realimentação movem o processo de fila conforme o comportamento.

Tempo real — taxas monotônicas (RMS: prioridade fixa, quanto menor o período maior a prioridade) e prazo mais curto primeiro (EDF: prioridade dinâmica).

Aprofundamento

O escalonador de curto prazo decide, entre os processos prontos, qual recebe a CPU. A primeira classificação é entre não preemptivo (o processo só larga a CPU por vontade própria — terminando ou bloqueando em E/S) e preemptivo (o SO pode retirá-lo, tipicamente por interrupção do temporizador). Todo sistema de tempo compartilhado é preemptivo; sem preempção, um processo em laço infinito travaria a máquina.

FCFS (First-Come, First-Served) ou FIFO atende na ordem de chegada, sem preempção. É trivial de implementar e justo na ordem, mas sofre o efeito comboio: um processo longo na frente faz todos os curtos esperarem, e o tempo médio de espera fica alto e imprevisível. Serve para sistemas em lote, não para interativos.

SJF (Shortest Job First) escolhe o processo de menor duração prevista. É comprovadamente ótimo para o tempo médio de espera — mover um processo curto para a frente de um longo reduz a espera do curto mais do que aumenta a do longo. Tem dois problemas: exige conhecer a duração de antemão, o que só se estima (tipicamente por média exponencial das rajadas anteriores), e causa inanição (starvation) dos processos longos se processos curtos continuarem chegando. A versão preemptiva, SRTF (Shortest Remaining Time First), reavalia a cada chegada: se o recém-chegado tem tempo restante menor que o do processo em execução, preempta-o. É o SRTF que gera o diagrama de Gantt da questão 6 do caderno.

Round-Robin (RR) é o FCFS com preempção por quantum: cada processo executa por até q unidades de tempo e, se não terminar, volta ao fim da fila. Com n processos e quantum q, nenhum processo espera mais que $(n - 1) \times q$ para voltar à CPU — é esse limite superior que o torna a escolha dos sistemas de tempo compartilhado. O quantum é o compromisso central: muito grande, o RR degenera em FCFS; muito pequeno, a troca de contexto passa a consumir fração relevante do tempo. Regra prática de Silberschatz: q deve ser grande em relação ao custo da troca de contexto, e cerca de 80% das rajadas de CPU devem ser menores que q. Os livros citam 10 a 100

ms; os sistemas atuais usam fatias bem menores — o escalonador do Linux trabalha com fatias-base da ordem de poucos milissegundos, ajustadas pela carga —, porque a troca de contexto ficou muito mais barata do que era quando essas faixas foram escritas.

Escalonamento por prioridade atribui um número a cada processo e escolhe o de maior prioridade; SJF é um caso particular, em que a prioridade é o inverso da duração. O problema é a inanição dos de baixa prioridade, e a solução clássica é o envelhecimento (aging): a prioridade de quem espera muito aumenta gradualmente. Filas múltiplas separam os processos por classe (sistema, interativos, lote), cada fila com seu algoritmo; filas múltiplas com realimentação movem processos entre as filas conforme o comportamento — quem usa a CPU demais desce, quem bloqueia com frequência sobe — e aproximam o SJF sem precisar prever durações.

O escalonador do Linux é um bom exemplo moderno. De 2007 a 2023 usou o CFS (Completely Fair Scheduler), que mantém para cada tarefa um tempo virtual de execução e sempre escolhe a de menor valor, aproximando um compartilhamento proporcional ideal. A partir do kernel 6.6 ele foi substituído pelo EEVDF (Earliest Eligible Virtual Deadline First), baseado em um artigo de 1995, que mantém a ideia de justiça proporcional mas acrescenta um prazo virtual a cada tarefa, escolhendo, entre as que ainda têm direito a CPU, a de prazo mais próximo — o que melhora a latência das tarefas interativas sem exigir os ajustes manuais do CFS. Para tempo real, os algoritmos clássicos são o Rate Monotonic (prioridade fixa, inversa ao período) e o EDF (Earliest Deadline First, prioridade dinâmica pelo prazo mais próximo).

Qual algoritmo para qual ambiente (Tanenbaum)

Tópico 5 · não existe algoritmo bom para todo ambiente — foi a questão do ENADE 2021

Síntese projetada

Sistemas em LOTE — ninguém espera no terminal: maximizar vazão e uso da CPU, sem preempção → FCFS, SJF (menor job primeiro) e SRTN.

Sistemas INTERATIVOS — usuário esperando: minimizar tempo de resposta, preempção obrigatória → Round-Robin, prioridades, múltiplas filas, loteria e fair-share.

Sistemas de TEMPO REAL — cumprir prazos e ser previsível → RMS (taxa monotônica, estático) e EDF (prazo mais curto primeiro, dinâmico); inversão de prioridade se resolve com herança de prioridade (ENADE 2008).

Objetivos comuns a todos — justiça, respeito à política e equilíbrio (CPU e E/S ocupadas). Os distratores de 2021 eram 'FILO para tempo real', 'RMS para lote', 'SJF para interativo' e 'RR para tempo real'.

Aprofundamento

Tanenbaum organiza os algoritmos pelo ambiente em que fazem sentido, e a questão 9 do ENADE 2021 pediu exatamente esse pareamento: um algoritmo seguido do ambiente a que ele é propício. Em lote não há usuário esperando, então vale maximizar a vazão e o uso da CPU sem preempção — FCFS, SJF e SRTN (o SJF preemptivo). Em sistemas interativos há alguém diante do terminal, e o critério passa a ser o tempo de resposta, com preempção obrigatória — Round-Robin, prioridades, múltiplas filas, escalonamento por loteria e fair-share (a resposta de 2021 era 'escalonamento por prioridades → sistemas interativos'). Em tempo real o objetivo é cumprir prazos de forma previsível — taxa monotônica (RMS), com prioridade fixa inversamente proporcional ao período, e prazo mais curto primeiro (EDF), com prioridade dinâmica. Comuns a todos: justiça, respeito à política do sistema e equilíbrio entre CPU e dispositivos. Em 2008 a prova discursiva cobrou a inversão de prioridade em tempo real e sua solução por herança de prioridade; em 2005, que o SJF não é o escalonador típico de tempo real.

O mesmo conjunto, três resultados

Tópico 5 · por que a escolha do algoritmo é decisão de projeto

Síntese projetada

Como calcular — espera = turnaround − duração; turnaround = conclusão − chegada. Monte o Gantt primeiro, depois some.

Aprofundamento

O exemplo da figura é o clássico de Silberschatz: três processos que chegam juntos em t = 0, com rajadas de 24, 3 e 3 ms. Ele foi escolhido porque escancara o efeito comboio e porque os números são simples de reproduzir à mão — o que é exatamente a habilidade que a prova mede.

Sob FCFS, P1 executa primeiro e os dois curtos esperam 24 e 27 ms; a média é 17 ms. Sob SJF, os curtos vão primeiro: P2 espera 0, P3 espera 3, P1 espera 6; a média cai para 3 ms — quase seis vezes menor, com o mesmo trabalho total. Sob Round-Robin com quantum 4, P1 executa 4 ms, cede a vez, P2 e P3 terminam em suas primeiras fatias, e P1 volta para concluir; a média é 5,7 ms — pior que SJF, mas com a garantia de que ninguém esperou mais que 7 ms para começar.

O procedimento para reproduzir qualquer um desses cálculos é sempre o mesmo, e convém ter um método fixo. Primeiro, desenhar a linha do tempo e marcar as chegadas. Segundo, a cada evento (chegada ou término), reavaliar a fila de prontos segundo a política. Terceiro, preencher o Gantt. Quarto, para cada processo, anotar o instante de conclusão e calcular turnaround = conclusão − chegada e espera = turnaround − duração. Quinto, tirar as médias. A maioria dos erros em prova vem de pular o segundo passo: aplicar a política uma vez e esquecer de reavaliá-la quando chega um novo processo.

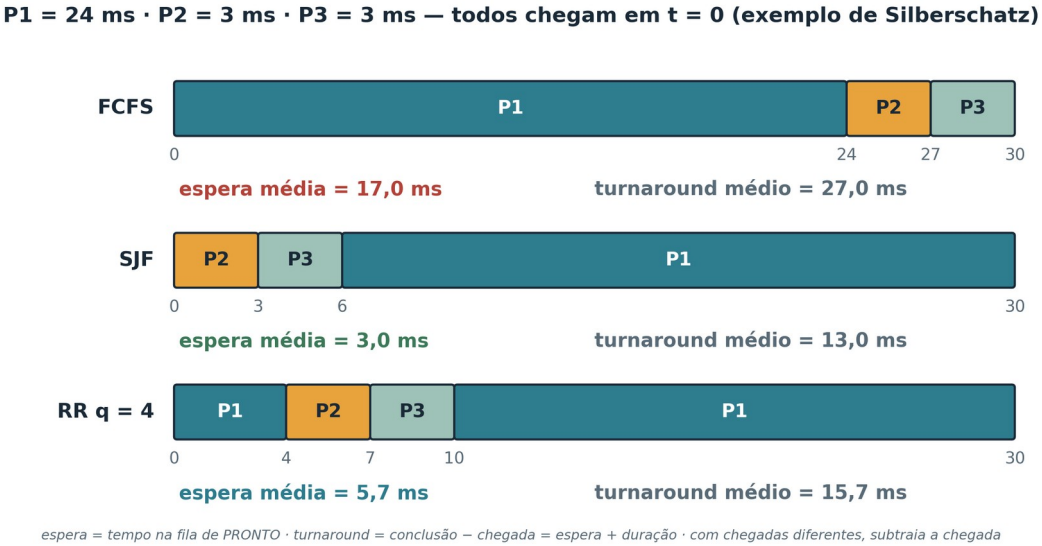


Figura 10 — Mesmo conjunto de processos sob FCFS, SJF e Round-Robin.

COMO LER A FIGURA

Cada linha é um diagrama de Gantt: o eixo horizontal é o tempo em ms, e cada barra colorida indica qual processo estava de posse da CPU naquele intervalo. Os números abaixo de cada linha são os instantes de troca. Abaixo de cada Gantt estão a espera média e o turnaround médio resultantes.

Compare as três linhas olhando para onde P1 (a barra grande) foi colocado: no início, no fim, ou fatiado. É essa posição que determina quanto os outros esperam. A frase de rodapé dá as duas fórmulas — espera = tempo na fila de prontos; turnaround = conclusão − chegada = espera + duração — e lembra de subtrair o instante de chegada quando os processos não chegam juntos. O ponto da figura: o mesmo hardware e a mesma carga produzem resultados muito diferentes conforme a política.

Concorrência: condição de corrida e região crítica

Tópico 4 · o problema que a sincronização resolve

Síntese projetada

Condição de corrida — o resultado depende da ordem em que os fluxos são escalonados. O mesmo código produz resultados diferentes a cada execução.

Região crítica — o trecho que acessa o recurso compartilhado.

As 4 condições de uma boa solução — 1) nunca dois na região crítica ao mesmo tempo; 2) nada suposto sobre velocidade ou nº de CPUs; 3) quem está fora não bloqueia ninguém; 4) ninguém espera para sempre.

Mecanismos — semáforo (contador + fila; bloqueia) · mutex (binário, com dono) · monitor (a linguagem faz o lock) · Peterson (só software) · TSL/test-and-set (instrução atômica; spinlock) · desabilitar interrupções (só o kernel, só em 1 CPU).

Aprofundamento

Concorrência é a situação em que dois ou mais fluxos de execução (processos ou threads) progridem ao mesmo tempo — de fato, em CPUs distintas, ou aparentemente, por revezamento em uma só. O problema surge quando eles acessam algo compartilhado: uma variável, um arquivo, uma estrutura de dados. Como o escalonador pode interromper um fluxo em qualquer instrução, o resultado passa a depender da ordem em que as instruções dos dois fluxos se intercalam — e essa ordem varia de execução para execução.

O exemplo canônico é o incremento de um contador compartilhado. A instrução 'x = x + 1' compila para três instruções de máquina: ler x em um registrador, somar 1, escrever de volta. Se a thread A é interrompida depois da leitura, a thread B lê o mesmo valor, incrementa e escreve; quando A retoma, escreve seu valor antigo incrementado, e um dos dois incrementos se perde. Isso é uma condição de corrida (race condition): o resultado depende de quem 'chega primeiro'. É um dos bugs mais difíceis de reproduzir porque só se manifesta com um entrelaçamento específico.

O trecho de código que acessa o recurso compartilhado é a região crítica (ou seção crítica). A solução é garantir exclusão mútua: no máximo um fluxo dentro da região crítica de cada vez. Tanenbaum enumera quatro condições para uma boa solução — e a prova as cobra: (1) dois processos nunca estão simultaneamente em suas regiões críticas; (2) nenhuma suposição é feita sobre velocidades ou número de CPUs; (3) nenhum processo fora de sua região crítica pode bloquear outro; (4) nenhum processo espera indefinidamente para entrar na região crítica.

Desabilitar interrupções resolve o problema em uma CPU — se o processo não pode ser interrompido, ninguém entra no meio — mas é inaceitável em modo usuário (instrução privilegiada; um programa poderia travar a máquina) e não funciona em múltiplas CPUs. A questão de 2021 do ENADE tratava exatamente disso. Soluções por software puro, como o algoritmo de Peterson, funcionam para dois processos com espera ocupada (busy waiting), desperdiçando CPU enquanto esperam. Soluções por hardware usam instruções atômicas — test-and-set, compare-and-swap — que leem e escrevem uma posição de memória em uma única operação indivisível, e são a base dos mecanismos de mais alto nível.

O semáforo, proposto por Dijkstra, é um contador inteiro com duas operações atômicas: down (P, wait, acquire) decrementa e bloqueia o chamador se o valor ficar negativo; up (V, signal, release) incrementa e acorda um processo bloqueado, se houver. Um semáforo inicializado em 1 funciona como mutex (semáforo binário): o primeiro a fazer down entra, os demais dormem até o up. Um semáforo inicializado em n controla n recursos idênticos. Ao contrário da espera ocupada, o processo bloqueado não consome CPU. Monitores são a mesma ideia empacotada pela linguagem (o synchronized do Java, por exemplo): a exclusão mútua é garantida pelo compilador, o que elimina a possibilidade de o programador esquecer o up.

Sincronização na prática

Tópico 4 · as ferramentas por baixo do semáforo — e o padrão que a prova repete

Síntese projetada

Desabilitar interrupções — impede a preempção enquanto a CPU está na região crítica. Só o kernel pode fazê-lo, e só funciona em MONOPROCESSADOR: outra CPU continua entrando. Foi a asserção-razão do ENADE 2021.

TSL / test-and-set e spinlock — instrução atômica de hardware que lê e escreve a variável de trava sem poder ser interrompida; a espera é OCUPADA (spin) — gasta CPU, mas é rápida para trechos curtos e funciona em multiprocessador.

Semáforo bloqueia; spinlock gira — down() sem crédito coloca o processo em ESPERA (sem gastar CPU); mutex = semáforo binário com dono; monitor = a linguagem faz o lock por você (synchronized em Java).

Produtor–consumidor (ENADE 2005 e 2014) — mutex = 1, vazio = N, cheio = 0. Produtor: down(vazio), down(mutex), insere, up(mutex), up(cheio). Consumidor: down(cheio), down(mutex), retira, up(mutex), up(vazio). Inverter down(mutex) com down(vazio) causa deadlock.

Aprofundamento

O problema do produtor-consumidor (ou do buffer limitado) é o exercício-padrão de semáforos e apareceu no ENADE de 2005 e de 2014. Um buffer com N posições é compartilhado por quem produz e por quem consome; é preciso impedir que o produtor insira com o buffer cheio, que o consumidor retire com o buffer vazio, e que os dois mexam na estrutura ao mesmo tempo. Três semáforos resolvem: 'vazio', iniciado em N, conta as posições livres; 'cheio', iniciado em 0, conta as ocupadas; 'mutex', iniciado em 1, protege a região crítica. O produtor faz down(vazio) — dorme se não há espaço —, depois down(mutex), insere, up(mutex) e up(cheio), acordando um consumidor que esperava; o consumidor é o espelho: down(cheio), down(mutex), retira, up(mutex), up(vazio). A ordem dos dois down() importa e é a pegadinha clássica: se o produtor fizer down(mutex) antes de down(vazio) e o buffer estiver cheio, ele dorme segurando o mutex — e o consumidor, que precisaria do mutex para esvaziar o buffer, nunca entra. É um deadlock causado por semáforos corretamente contados mas adquiridos na ordem errada, o mesmo mecanismo da questão de 2017.

Deadlock: as quatro condições de Coffman

Tópico 4 · exclusão mútua não previne impasse — ela é uma das causas

Síntese projetada

Tratamento — PREVENIR (quebrar uma condição, p. ex. impor ordem global de aquisição) · EVITAR (algoritmo do banqueiro) · DETECTAR e recuperar · IGNORAR (avestruz — o que Linux e Windows fazem para os recursos das aplicações).

Aprofundamento

Deadlock (impasse) é a situação em que um conjunto de processos está bloqueado, cada um esperando um recurso que só outro processo do conjunto pode liberar — e nenhum libera, porque todos estão esperando. Não é um bug de lógica no sentido usual: cada processo, isoladamente, está fazendo algo razoável. O impasse é uma propriedade do conjunto.

Coffman e colegas (1971) mostraram que quatro condições precisam valer simultaneamente para que um deadlock ocorra. Exclusão mútua: o recurso só pode ser usado por um processo de cada vez. Posse e espera (hold and wait): um processo que já detém recursos pode solicitar outros sem liberar os que tem. Não preempção: um recurso só é liberado voluntariamente por quem o detém; o SO não o toma à força. Espera circular: existe uma cadeia fechada P1 → P2 → … → Pn → P1 em que cada um espera um recurso detido pelo seguinte. Se qualquer uma das quatro falhar, o deadlock é impossível.

O ponto conceitual mais importante do tópico — e o que a questão de asserção-razão explora — é que exclusão mútua não previne o impasse: ela é a primeira das quatro condições que o tornam possível. Um sistema pode usar semáforos de forma impecável, sem nenhuma condição de corrida, e ainda assim travar. Condição de corrida e deadlock são problemas distintos: o primeiro é acesso simultâneo sem coordenação; o segundo é coordenação que trava. O semáforo resolve o primeiro e, mal usado, causa o segundo.

O cenário clássico é o de dois mutexes adquiridos em ordens opostas: a thread T1 pega A e depois pede B; a thread T2 pega B e depois pede A. Se o escalonador intercalar as aquisições — T1 pega A, T2 pega B — cada uma fica esperando o mutex que a outra detém, para sempre. Foi essa a questão do ENADE 2017. O grafo de alocação de recursos torna o diagnóstico visual: processos são círculos, recursos são retângulos, uma seta de recurso para processo significa 'detém', de processo para recurso significa 'pede'. Um ciclo no grafo é condição necessária para deadlock (e suficiente quando cada recurso tem uma única instância).

Há quatro estratégias de tratamento. Prevenção: quebrar estruturalmente uma das quatro condições — a mais prática é quebrar a espera circular impondo uma ordem global de aquisição (todo mundo pede A antes de B, nunca o contrário), ou quebrar posse e espera exigindo que o processo peça todos os recursos de uma vez. Evitação (avoidance): conceder cada pedido só se o sistema permanecer em estado seguro, isto é, se ainda existir uma ordem em que todos consigam terminar — é o algoritmo do banqueiro, de Dijkstra, que exige conhecer de antemão o máximo que cada processo vai pedir. Detecção e recuperação: deixar acontecer, procurar ciclos periodicamente e resolver matando um processo ou tomando um recurso. Ignorar: o chamado algoritmo do avestruz — assumir que o impasse é raro o bastante para não valer o custo da prevenção, e deixar para o usuário reiniciar. É o que Linux e Windows fazem em relação aos recursos das aplicações, delegando a responsabilidade a elas. Dentro do próprio núcleo a postura é outra: o Linux tem o lockdep, um validador que, em kernels de depuração, verifica a ordem de aquisição dos locks e detecta potenciais impasses antes que ocorram — na prática, prevenção por ordem global, aplicada ao código do kernel.

Distinga também deadlock de inanição (starvation) e de livelock. Na inanição, o processo não avança porque outros são sempre preferidos — mas poderia avançar se o escalonador quisesse; não há espera circular. No livelock, os processos mudam de estado continuamente sem progredir — estão 'vivos' mas presos. No deadlock, estão bloqueados e nenhuma decisão de escalonamento os libera.

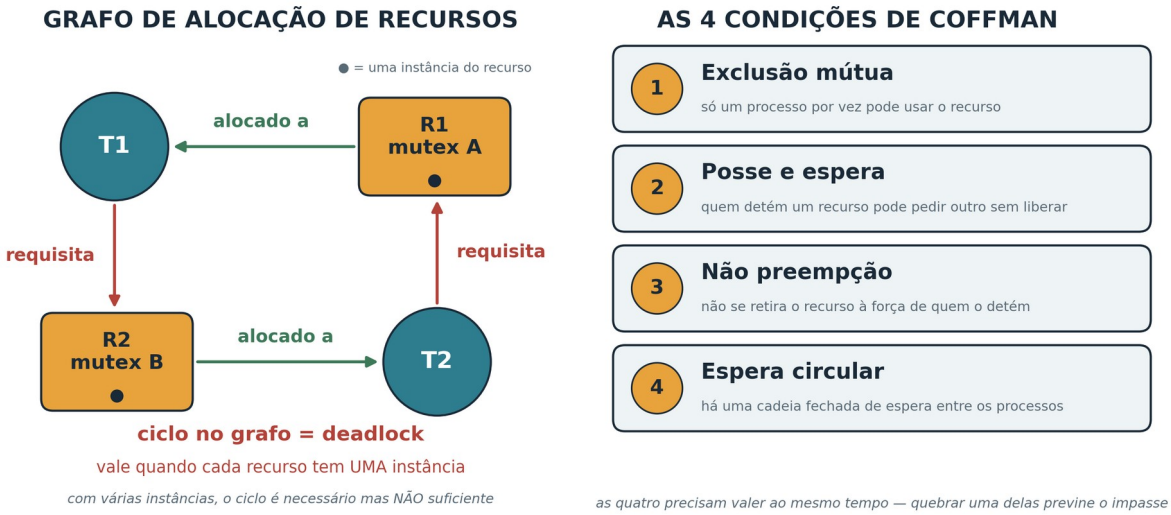


Figura 9 — Grafo de alocação com espera circular e as condições de Coffman.

COMO LER A FIGURA

À esquerda, o grafo de alocação de recursos do cenário de dois mutexes: T1 detém R1 (seta verde de R1 para T1) e espera R2 (seta vermelha de T1 para R2); T2 detém R2 e espera R1. Siga as setas: T1 → R2 → T2 → R1 → T1. O ciclo está fechado, e é ele que caracteriza o impasse — o ponto dentro de cada retângulo indica que o recurso tem uma única instância, e só nesse caso o ciclo é condição suficiente.

À direita, as quatro condições de Coffman numeradas. A leitura correta é a da frase do rodapé: as quatro precisam valer juntas, e quebrar uma delas já previne. No cenário da esquerda, a mais fácil de quebrar é a quarta — bastaria que T2 pedisse R1 antes de R2, na mesma ordem que T1.

Tratamento de deadlock: quatro estratégias

Tópico 4 · o que fazer com o impasse

Síntese projetada

Avestruz (ignorar) — assume que é raro e que conviver sai mais barato que prevenir; é o que Linux e Windows fazem para a maioria dos recursos.

Prevenção — projetar o sistema para negar UMA das quatro condições. A mais usada: ordem global de aquisição (todos pegam A antes de B) quebra a espera circular — foi a resposta do ENADE 2017.

Evitar (avoidance) — decidir a cada pedido se o sistema continua em ESTADO SEGURO — algoritmo do banqueiro; exige saber a necessidade máxima de cada processo, o que é raro na prática.

Detecção e recuperação — deixar acontecer, detectar (ciclo no grafo ou matriz de alocação) e recuperar: abortar um processo, preemptar um recurso ou voltar a um ponto de verificação (rollback).

Termos que a prova usa

VOCABULÁRIO DA SEÇÃO 2

Processo — programa em execução; unidade de posse de recursos.

PCB — estrutura do núcleo que representa o processo (contextos de hardware e software, espaço de endereçamento).

Troca de contexto — salvar o estado de um processo e restaurar o de outro.

Thread — fluxo de execução dentro de um processo; tem PC, registradores e pilha próprios.

Pronto / executando / esperando — os três estados centrais do ciclo de vida.

Preempção — retirada da CPU de um processo que não terminou.

Dispatcher — componente que efetiva a troca de contexto decidida pelo escalonador.

Turnaround, espera, resposta — conclusão − chegada; tempo na fila de prontos; até a primeira resposta.

Quantum — fatia de tempo do Round-Robin.

Efeito comboio — processos curtos presos atrás de um longo no FCFS.

Inanição (starvation) — processo indefinidamente preterido pelo escalonador.

Condição de corrida — resultado que depende da ordem de intercalação dos fluxos.

Região crítica — trecho que acessa recurso compartilhado.

Exclusão mútua — no máximo um fluxo na região crítica por vez.

Semáforo, mutex, monitor — contador com down/up atômicos; semáforo binário; construção da linguagem.

Deadlock — conjunto de processos bloqueados esperando uns pelos outros.

Condições de Coffman — exclusão mútua, posse e espera, não preempção, espera circular.

Erros mais comuns

ONDE OS ESTUDANTES ERRAM

- Desenhar a transição esperando → executando. Não existe: sempre se passa por pronto.
- Aplicar a política de escalonamento uma só vez, sem reavaliar a fila a cada chegada. É o que faz o SRTF virar SJF não preemptivo na resposta errada.
- Confundir tempo de espera com turnaround. Espera = turnaround − duração.
- Achar que exclusão mútua previne deadlock. Ela é a primeira condição de Coffman.
- Confundir deadlock com inanição. Na inanição não há espera circular; o processo poderia avançar.
- Tratar thread como 'processo pequeno'. Thread compartilha o espaço de endereçamento; processo não.

Questões desta seção

Consulte o caderno de questões: questões 4, 5 e 6. O gabarito comentado, com a análise de cada distrator, está no documento de gabarito.

Seção 3 — Gerência de Memória Principal

Tópico 6 — hierarquia, particionamento, fragmentação e relocação

- T6 · Memória principal: alocação contígua, particionamento, endereçamento e relocação

A gerência de memória é o segundo grande recurso que o SO administra, e a Matriz de Referência a divide em dois tópicos que aqui viram duas seções: a memória principal como recurso físico (T6, esta seção) e a memória virtual como abstração sobre ela (T7, a próxima). Esta seção estabelece o vocabulário — hierarquia, localidade, particionamento, fragmentação, endereço lógico e físico, relocação — sem o qual a memória virtual não faz sentido.

Historicamente, o que se cobra aqui é a distinção entre fragmentação interna e externa, as estratégias de alocação (first, best, worst-fit) e a diferença entre endereço lógico e físico. São conceitos de fundamentação, mas com armadilhas de vocabulário que o ENADE explora.

Hierarquia de memória

Tópico 6 · cinco níveis — e o princípio da localidade que torna a hierarquia viável

Síntese projetada

> **Localidade de referência —** temporal (o que foi acessado tende a ser acessado de novo) e espacial (vizinhos tendem a ser acessados). Sem ela, cache e memória virtual não funcionariam.

Aprofundamento

Toda a gerência de memória parte de um fato físico: memória rápida é cara e pequena; memória grande é barata e lenta. Registradores respondem em um ciclo de clock, mas há poucas dezenas deles; a cache responde em alguns ciclos e tem megabytes; a RAM responde em uma centena de ciclos e tem gigabytes; o disco responde em milhões de ciclos e tem terabytes. A hierarquia de memória é a solução de engenharia para essa tensão: organizar os níveis em camadas e manter em cada camada o que está mais perto de ser usado.

O que torna isso possível é o princípio da localidade de referência, uma propriedade empírica dos programas. Localidade temporal: o que foi acessado há pouco tende a ser acessado de novo em breve (variáveis de laço, topo da pilha). Localidade espacial: endereços próximos ao que foi acessado tendem a ser acessados em seguida (instruções sequenciais, elementos de um vetor). Graças à localidade, uma cache pequena consegue atender a grande maioria dos acessos — e o mesmo raciocínio, um nível abaixo, é o que permite à memória virtual manter no disco a maior parte do espaço de endereçamento sem que o programa perceba.

Cada nível funciona como cache do nível abaixo, e o gerenciamento se divide: registradores são administrados pelo compilador; a cache pelo hardware, de forma transparente ao SO; a memória principal e o disco pelo sistema operacional — é aqui que entra a gerência de memória propriamente dita. A métrica que resume o desempenho da hierarquia é o tempo médio de acesso: se a taxa de acerto no nível rápido é h, o tempo de acesso ao nível rápido é t1 e o de falha (buscar no nível lento) é t2, o tempo médio é h·t1 + (1 − h)·t2. Com t2 milhares de vezes maior que t1, mesmo uma taxa de acerto de 99% pode ser dominada pelo termo das falhas — daí a importância de algoritmos que escolham bem o que manter em cada nível.

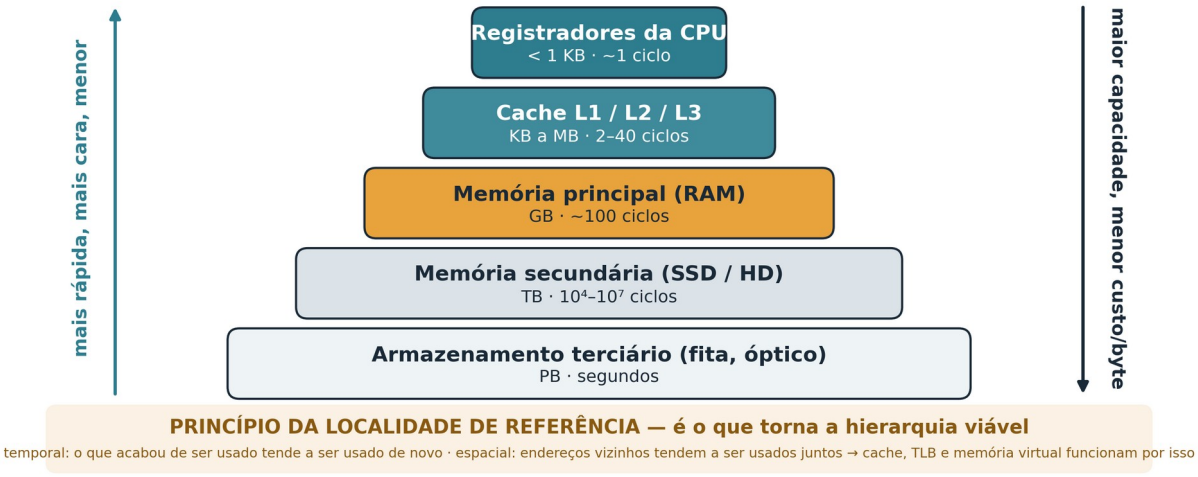


Figura 11 — Hierarquia de memória.

COMO LER A FIGURA

A pirâmide é lida em duas direções. Subindo (seta da esquerda): mais rápido, mais caro por byte, menor capacidade. Descendo (seta da direita): maior capacidade, menor custo por byte, mais lento. Em cada camada estão a ordem de grandeza da capacidade e do tempo de acesso em ciclos — repare que da RAM para o disco o salto é de quatro a cinco ordens de grandeza, muito maior que qualquer outro degrau.

A frase de rodapé nomeia o que sustenta a pirâmide inteira: a localidade de referência. Sem ela, nenhum nível funcionaria como cache do de baixo, e toda a hierarquia seria inútil.

Particionamento e os dois tipos de fragmentação

Tópico 6 · a distinção mais cobrada do tópico

Síntese projetada

Decore assim — INTERNA = desperdício DENTRO da área alocada (partições fixas). EXTERNA = desperdício FORA, em lacunas dispersas (partições variáveis).

Aprofundamento

Na alocação contígua, cada processo ocupa uma única faixa contínua de endereços físicos. É o esquema mais simples e o que dá origem ao vocabulário de fragmentação que a prova cobra. Há dois modos de fazê-lo: dividir a memória previamente em partições de tamanho fixo, ou criar partições sob demanda com o tamanho exato de cada processo.

No particionamento fixo (estático), a memória é dividida na inicialização em partições de tamanhos definidos — todas iguais, ou de tamanhos variados. Um processo é carregado em uma partição grande o bastante para ele; o que sobrar dentro da partição fica inutilizado, porque a partição não pode ser dividida. Esse espaço perdido dentro da área alocada é a fragmentação interna. Ela é previsível e limitada pelo tamanho das partições, mas em média desperdiça metade de uma partição por processo, e limita o grau de multiprogramação ao número de partições.

No particionamento variável (dinâmico), o SO mantém uma lista de áreas livres e, ao carregar um processo, recorta uma área do tamanho exato. Não há fragmentação interna. Mas à medida que processos terminam e outros são carregados, as áreas livres vão sendo fatiadas em pedaços cada vez menores, espalhados pela memória. Chega um momento em que a soma dos espaços livres é suficiente para um novo processo, mas

nenhum deles, isoladamente, é grande o bastante. Esse desperdício fora das áreas alocadas, em lacunas dispersas, é a fragmentação externa.

A regra para não confundir: fragmentação interna é dentro do que foi alocado, e decorre de alocar mais do que o processo pediu (partições fixas; também blocos de disco e páginas, como se verá); fragmentação externa é fora do alocado, e decorre de o espaço livre estar em pedaços não contíguos (partições variáveis, segmentação, alocação contígua em disco). A paginação elimina a externa e reintroduz uma interna pequena na última página de cada processo.

A solução clássica para a fragmentação externa é a compactação: mover os processos para juntar todas as lacunas em uma única área livre contígua. É cara — copia todo o conteúdo da memória — e só é possível com relocação dinâmica, porque os endereços dos processos mudam. A solução definitiva é abandonar a exigência de contiguidade, que é o que a paginação faz.

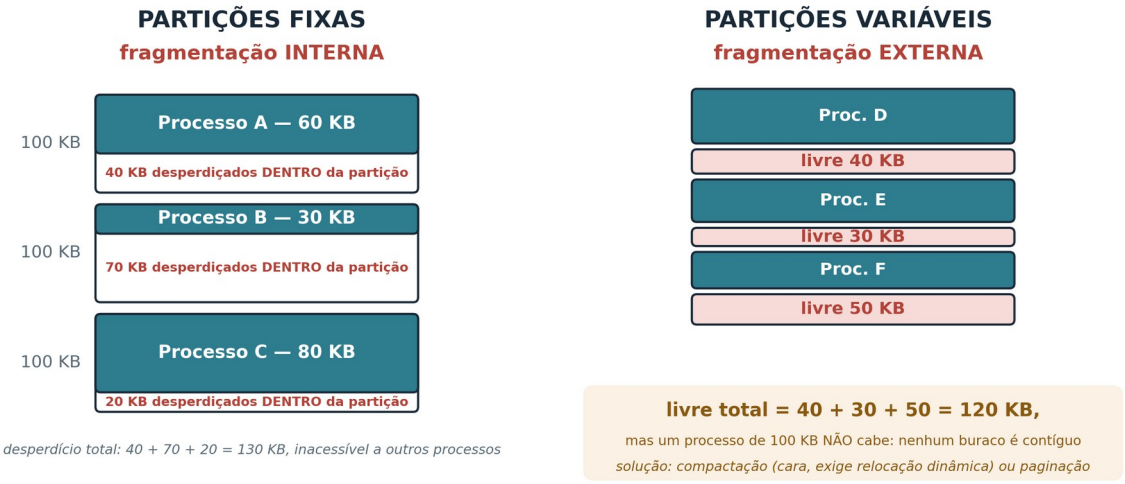


Figura 12 — Fragmentação interna e externa.

COMO LER A FIGURA

À esquerda, três partições fixas de 100 KB ocupadas por processos menores. A parte branca de cada partição, marcada em vermelho, é a fragmentação interna: espaço reservado, mas sem uso, dentro da partição. Some os três desperdícios: 130 KB que nenhum outro processo pode usar, porque estão dentro de partições já atribuídas.

À direita, partições variáveis: cada processo ocupou exatamente o que pediu, e as áreas livres (em rosa) ficaram espalhadas entre eles. Há 120 KB livres no total, mas o maior pedaço tem 50 KB — um processo de 100 KB não cabe em lugar nenhum. Isso é a fragmentação externa. O rodapé da coluna esquerda soma o desperdício interno; o da direita dá a solução da externa que a prova espera: compactação (cara, e só possível com relocação dinâmica) ou abandonar a contiguidade com paginação.

Estratégias de alocação

Tópico 6 · onde colocar o processo que acabou de chegar

Síntese projetada

First-fit — a primeira lacuna que couber. É a mais rápida, porque para de procurar assim que encontra.

Best-fit — a MENOR lacuna que couber. Deixa o menor resto, mas exige varrer a lista inteira e espalha lacunas pequenas demais para reusar.

Worst-fit — a MAIOR lacuna de todas, com o argumento de deixar um resto grande e reaproveitável. Na prática, desempenho ruim.

Compactação — junta as áreas livres num único bloco contíguo. Resolve a fragmentação externa, mas custa caro e exige relocação dinâmica.

Aprofundamento

Quando um processo chega e há várias áreas livres onde ele caberia, alguém tem de escolher. As estratégias de alocação são as regras dessa escolha, e a prova costuma dar um mapa de áreas livres e pedir onde cada estratégia colocaria um pedido — como na questão 7 do caderno.

First-fit escolhe a primeira área livre, na ordem dos endereços, que seja grande o bastante. É a mais rápida, porque para de procurar ao encontrar. Tende a concentrar alocações no início da memória e a acumular pequenos restos ali, o que a variante next-fit (continuar a busca de onde a anterior parou) tenta amenizar. Best-fit escolhe a menor área livre que ainda comporte o pedido, deixando o menor resto possível. Exige percorrer toda a lista (ou mantê-la ordenada) e tem um efeito colateral contraintuitivo: os restos que deixa são minúsculos demais para qualquer uso, o que agrava a fragmentação externa. Worst-fit escolhe a maior área de todas, com o argumento de que o resto será grande e reaproveitável; na prática, consome rapidamente as áreas grandes e tem o pior desempenho nas simulações.

As simulações clássicas (Knuth, reproduzidas em Silberschatz) indicam que first-fit e best-fit são melhores que worst-fit em tempo e em aproveitamento de espaço, e que first-fit é geralmente mais rápido. Nenhuma das três resolve a fragmentação externa; todas apenas a administram. A regra dos 50%, empírica, diz que com first-fit, para N blocos alocados há cerca de 0,5N blocos perdidos por fragmentação — um terço da memória.

Endereço lógico, endereço físico e relocação

Tópico 6 · como o processo roda em qualquer posição da memória

Síntese projetada

Relocação estática × dinâmica — estática: os endereços são fixados na carga e o processo não pode ser movido. Dinâmica: a MMU traduz a cada acesso, e o processo pode ser movido ou levado para o disco.

Swapping — levar um processo inteiro da memória para a área de troca no disco e trazê-lo de volta depois, liberando espaço para outros.

Aprofundamento

Um programa compilado contém endereços — de suas variáveis, de suas funções, de seus saltos. Mas o compilador não sabe em que posição da memória física o programa será carregado, e o programa pode ser carregado em posições diferentes a cada execução, ou até movido durante ela. A solução é separar dois espaços: o endereço lógico (ou virtual), gerado pela CPU ao executar o programa, e o endereço físico, que de fato chega ao barramento de memória. A tradução entre eles é feita por hardware — a MMU, unidade de gerência de memória — e configurada pelo SO.

O esquema mais simples de tradução é o de registradores base e limite. O registrador base contém o endereço físico onde o processo começa; o limite contém o tamanho do processo. A cada acesso, a MMU compara o endereço lógico com o limite — se for maior ou igual, gera uma exceção de endereçamento, e o processo é encerrado por tentar sair do seu espaço — e, se estiver dentro, soma a base para obter o endereço físico. Dois registradores por processo, carregados pelo dispatcher na troca de contexto, dão ao mesmo tempo relocação e proteção.

Relocação é o nome do ato de ajustar os endereços de um programa à posição em que ele foi carregado. Na relocação estática, isso é feito uma vez, no momento da carga: o carregador soma o endereço inicial a cada referência do código, e o programa passa a conter endereços físicos. O processo não pode mais ser movido — mover exigiria recarregá-lo. Na relocação dinâmica, o código mantém endereços lógicos e a MMU traduz a cada acesso, em tempo de execução. Mover o processo custa apenas copiar seu conteúdo e atualizar o registrador

base. É a relocação dinâmica que torna possíveis a compactação, o swapping e, generalizando de um registrador base para uma tabela de páginas, a memória virtual.

Swapping é a técnica de retirar um processo inteiro da memória, gravá-lo em uma área de troca no disco (swap) e trazê-lo de volta depois, quando houver espaço ou quando ele voltar a ser escalonado. Foi a primeira forma de executar mais processos do que a memória comporta, e sobrevive em sistemas modernos como último recurso quando a memória virtual por páginas não basta. Seu custo é alto — transferir dezenas de megabytes para o disco leva dezenas de milissegundos. Um cuidado de vocabulário: nos sistemas atuais a palavra 'swap' designa a área de disco para onde a memória virtual leva PÁGINAS individuais, não processos inteiros. O swapping clássico, de processo inteiro, é o modelo dos livros e sobrevive apenas como último recurso; o que acontece no dia a dia é paginação para disco.

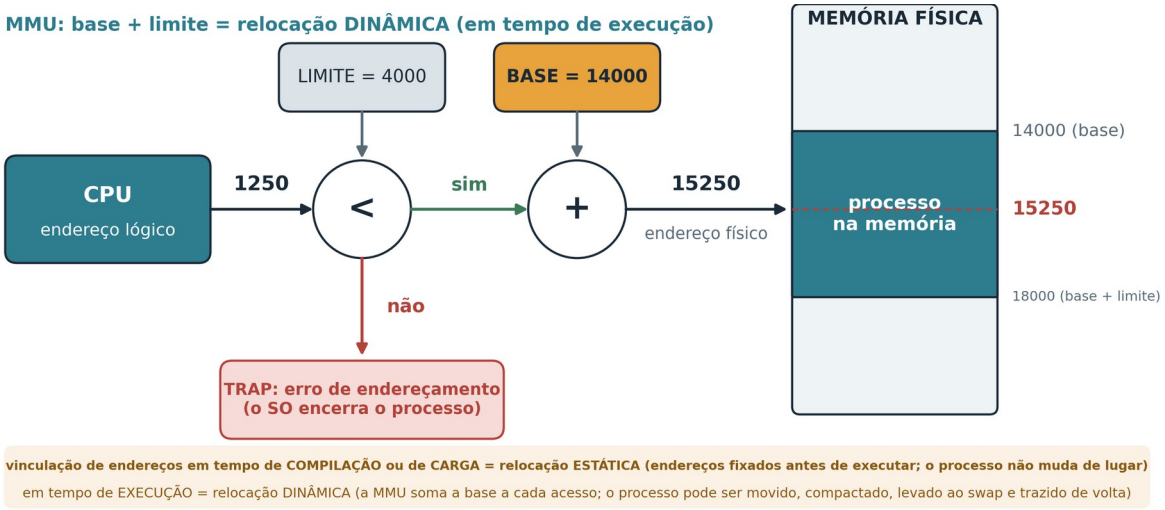


Figura 14 — Tradução por registradores base e limite na MMU.

COMO LER A FIGURA

Siga o endereço lógico 1250 gerado pela CPU. Ele passa primeiro pelo comparador com o registrador LIMITE (4000): como 1250 < 4000, o acesso é válido e segue para o somador com o registrador BASE (14000); se fosse maior ou igual ao limite, o hardware geraria um trap. A soma dá 15250, que é o endereço físico, dentro da faixa 14000–18000 que o processo ocupa na memória.

Se o endereço lógico fosse 4500, o comparador dispararia a exceção de endereçamento (seta vermelha) e o acesso jamais chegaria à memória — é a proteção. A frase de rodapé grava o conceito: isso é relocação dinâmica, feita pela MMU a cada acesso, em tempo de execução.

Termos que a prova usa

VOCABULÁRIO DA SEÇÃO 3

Hierarquia de memória — registradores → cache → RAM → disco, do mais rápido ao maior.

Localidade temporal / espacial — reuso próximo no tempo / acesso a vizinhos.

Alocação contígua — processo em uma única faixa contínua de memória física.

Partição fixa / variável — definida na inicialização / criada sob demanda.

Fragmentação interna — desperdício dentro da área alocada.

Fragmentação externa — desperdício em lacunas livres dispersas, não contíguas.

First-fit / best-fit / worst-fit — primeira que cabe / menor que cabe / maior de todas.

Compactação — mover processos para unir as lacunas livres.

Endereço lógico / físico — gerado pela CPU / entregue à memória.

MMU — hardware que traduz lógico em físico.

Base e limite — registradores que dão relocação e proteção.

Relocação estática / dinâmica — na carga, uma vez / a cada acesso, pela MMU.

Swapping — levar um processo inteiro para o disco e trazê-lo de volta.

Erros mais comuns

ONDE OS ESTUDANTES ERRAM

- Atribuir fragmentação interna às partições variáveis. Nelas a partição tem o tamanho exato do processo; o problema é a externa.
- Confundir best-fit com first-fit no cálculo. Best-fit exige olhar todas as áreas e escolher a menor que cabe.
- Achar que compactação resolve a fragmentação interna. Ela resolve a externa, e só com relocação dinâmica.
- Tratar endereço lógico e físico como sinônimos. O programa só vê lógicos; a memória só recebe físicos.

Questões desta seção

Consulte o caderno de questões: questão 7. O gabarito comentado, com a análise de cada distrator, está no documento de gabarito.

Seção 4 — Memória Virtual

Tópico 7 — paginação, segmentação, TLB, substituição e thrashing

- T7 · Memória virtual: paginação, segmentação, mapeamento e substituição de páginas

A memória virtual é o tópico da Matriz de Referência com mais questões históricas em provas de Computação — o ENADE 2017 trouxe FIFO × LRU em asserção-razão, o de 2014 uma tradução de endereço com quadros de 1 KB e bit de validade, e o de 2005 a afirmação de que o espaço virtual pode ser maior que a memória física — e é também o que mais exige cálculo: simular uma cadeia de referências, contar faltas de página, calcular tempo efetivo de acesso. É um tópico em que saber a teoria não basta; é preciso ter um procedimento mecânico e treinado.

A fala deve seguir a lógica do mecanismo: por que queremos executar programas maiores que a memória; como a paginação traduz endereços; por que a TLB é indispensável; o que acontece numa falta de página; como escolher a vítima; e o que dá errado quando há processos demais — o thrashing.

A ideia central da memória virtual

Tópico 7 · executar o que não cabe

Síntese projetada

O espaço lógico é maior que o físico — só as partes em uso ficam na memória principal; o resto espera no disco.

Some a fragmentação externa — o processo deixa de precisar de espaço contíguo — os quadros podem estar em qualquer lugar.

Preço a pagar — cada acesso passa por uma tradução, e uma referência a página ausente custa uma operação de disco — milhões de vezes mais lenta que um acesso à RAM.

Aprofundamento

A memória virtual leva a separação entre endereço lógico e físico à sua conclusão natural: se o programa só vê endereços lógicos e a MMU traduz cada um deles, então o espaço de endereçamento lógico não precisa caber inteiro na memória física — e nem precisa estar todo lá ao mesmo tempo. As partes em uso ficam na RAM; as demais ficam no disco e são trazidas sob demanda, quando referenciadas. Para o programa, é como se houvesse uma memória enorme e toda sua.

Isso resolve três problemas de uma vez. Programas maiores que a memória física passam a executar. Mais processos cabem simultaneamente na memória, porque cada um ocupa só a parte que está usando — o grau de multiprogramação aumenta. E, como as partes de um processo podem estar em qualquer lugar da memória física, a exigência de contiguidade desaparece e com ela a fragmentação externa.

O que torna tudo isso viável, mais uma vez, é a localidade de referência: em qualquer intervalo curto, um programa usa apenas uma fração pequena de seu espaço de endereçamento. Se não fosse assim, o SO passaria o tempo todo trazendo e levando partes do disco, e o sistema seria inutilizável — que é exatamente o que acontece no thrashing.

O preço é duplo. Cada acesso à memória agora passa por uma tradução, o que sem apoio de hardware dobraria o tempo de acesso (daí a TLB). E uma referência a uma parte que não está na RAM custa uma operação de disco: se um acesso à RAM leva 100 ns e uma falta de página leva 8 ms, a falta é 80 mil vezes mais lenta. Uma taxa de falta de apenas 1 em mil já multiplica o tempo médio de acesso por 80. Por isso a memória virtual só funciona bem se a taxa de faltas for baixíssima — e todo o resto da seção é sobre como mantê-la baixa.

Paginação: como o endereço é traduzido

Tópico 7 · página, quadro, deslocamento e bits de controle

Síntese projetada

Endereço lógico = (p, d) — p indexa a tabela de páginas e devolve o quadro f; d é copiado sem alteração para o endereço físico.

Aprofundamento

A paginação divide o espaço de endereçamento lógico em blocos de tamanho fixo chamados páginas, e a memória física em blocos do mesmo tamanho chamados quadros (frames). O tamanho típico é 4 KB, mas processadores modernos suportam páginas grandes (2 MB, 1 GB). Qualquer página pode ser colocada em qualquer quadro; o mapeamento de uma para o outro é registrado na tabela de páginas, uma por processo, mantida pelo SO e consultada pela MMU.

Um endereço lógico é dividido em duas partes: os bits mais significativos formam o número da página (p), e os menos significativos, o deslocamento (d) dentro da página. Com páginas de 4 KB (2^{12} bytes), o deslocamento tem 12 bits; em um endereço de 32 bits, sobram 20 para o número da página, o que dá uma tabela de até 2^{20} = 1 milhão de entradas por processo. A tradução é: usar p como índice na tabela de páginas, obter o número do quadro f, e formar o endereço físico concatenando f com o mesmo d. O deslocamento não é traduzido — passa direto, porque página e quadro têm o mesmo tamanho.

Cada entrada da tabela de páginas carrega, além do número do quadro, bits de controle que o hardware e o SO usam. O bit de validade (ou presença) diz se a página está na memória; se estiver zerado, a MMU gera a exceção de falta de página. O bit de referência (R) é ligado pelo hardware a cada acesso, e o SO o usa para saber que páginas estão em uso — é a matéria-prima dos algoritmos de substituição. O bit de modificação (M, ou dirty) é ligado quando a página é escrita; se estiver zerado quando a página for escolhida como vítima, ela não precisa ser gravada de volta no disco, porque a cópia de lá continua idêntica — economiza uma operação de E/S inteira. Há ainda bits de proteção (leitura, escrita, execução) e de cache.

O problema prático da paginação é o tamanho da tabela. Um milhão de entradas de 4 bytes é 4 MB por processo — só para a tabela — e em 64 bits o espaço lógico é tão grande que uma tabela linear é impossível. A solução dos sistemas reais é a tabela de páginas multinível (hierárquica): o número da página é dividido em vários campos, cada um indexando um nível, e só os níveis correspondentes a regiões efetivamente usadas do espaço de endereçamento são alocados. O x86-64 usa quatro ou cinco níveis. A alternativa é a tabela de páginas invertida, com uma entrada por quadro físico em vez de por página lógica.

A paginação não tem fragmentação externa — qualquer quadro livre serve para qualquer página — mas tem uma fragmentação interna pequena: a última página de cada processo é, em média, meio vazia. Com páginas de 4 KB, isso é irrisório; é um dos motivos pelos quais páginas pequenas são preferidas, contrabalançado pelo fato de que páginas menores significam tabelas maiores e mais faltas de TLB.

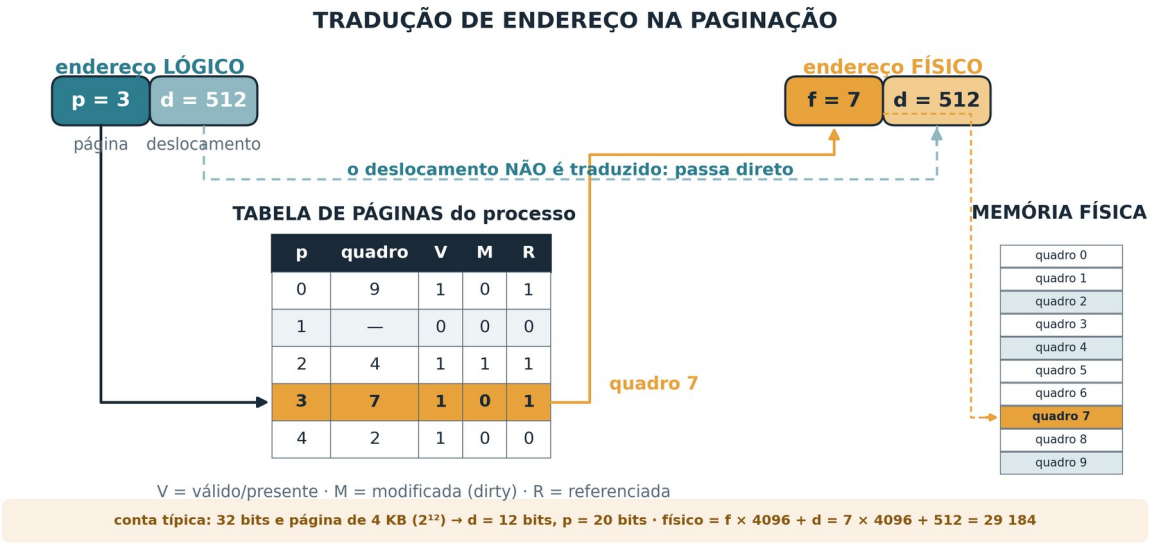


Figura 15 — Tradução de endereço na paginação.

COMO LER A FIGURA

Siga o endereço lógico (p = 3, d = 512) da esquerda para a direita. O número da página, 3, entra na tabela de páginas do processo (seta preta) e seleciona a linha destacada em laranja, que informa o quadro 7 — com bit V = 1 (presente), M = 0 (não modificada) e R = 1 (referenciada). O número do quadro segue pela seta laranja para compor o endereço físico. O deslocamento 512 segue pela seta tracejada e é copiado sem alteração: é a linha 'o deslocamento não é traduzido'.

Repare na linha p = 1 da tabela, com V = 0 e quadro '—': é uma página que não está na memória. Um acesso a ela dispararia a falta de página descrita duas seções adiante. O rodapé traz a conta típica de prova: endereço de 32 bits e páginas de 4 KB → 12 bits de deslocamento e 20 de página; físico = f × 4096 + d. À direita, a memória física mostra o quadro 7 em destaque, para lembrar que o quadro é um pedaço da RAM e que o processo não faz ideia de qual quadro está usando.

Segmentação e segmentação paginada

Tópico 7 · dividir por significado, não por tamanho fixo

Síntese projetada

Paginação — blocos de TAMANHO FIXO, divisão física. Gera fragmentação INTERNA (na última página) e nenhuma externa.

Segmentação — blocos de TAMANHO VARIÁVEL com significado lógico (código, pilha, dados). Facilita proteção e compartilhamento, mas reintroduz a fragmentação EXTERNA.

Segmentação paginada — segmentos divididos em páginas — combina proteção lógica com ausência de fragmentação externa. Foi o modelo do IA-32; em x86-64 a segmentação é vestigial e prevalece a paginação multinível (4 ou 5 níveis).

Aprofundamento

A paginação divide a memória por tamanho, ignorando o significado do que está em cada página. A segmentação divide por significado: o espaço de endereçamento é um conjunto de segmentos de tamanhos variáveis, cada um correspondendo a uma unidade lógica do programa — o código, a pilha, o heap, uma tabela de símbolos, uma biblioteca compartilhada. O endereço lógico passa a ser um par (segmento, deslocamento), e a tabela de segmentos guarda, para cada um, a base física e o limite.

As vantagens vêm do significado. Como o segmento de código é uma unidade, pode ser marcado como somente-leitura e executável, e o de dados como leitura-escrita e não executável — proteção com granularidade lógica. Como uma biblioteca é um segmento, pode ser mapeada no espaço de vários processos e compartilhada com facilidade. E como cada segmento cresce independentemente, a pilha e o heap podem expandir sem colidir. A desvantagem é que segmentos têm tamanhos variáveis e precisam ser contíguos na memória física — o que reintroduz a fragmentação externa, exatamente o problema que a paginação havia eliminado.

A segmentação paginada combina os dois: o espaço é dividido em segmentos, e cada segmento é dividido em páginas. O endereço lógico tem três partes (segmento, página, deslocamento); a tabela de segmentos aponta para uma tabela de páginas por segmento. Obtém-se a proteção e o compartilhamento lógicos da segmentação com a ausência de fragmentação externa da paginação. O Intel x86 de 32 bits implementou exatamente esse esquema; em 64 bits a segmentação foi praticamente abandonada e a proteção é feita por bits nas entradas da tabela de páginas, mas o conceito continua sendo cobrado.

TLB: sem ela cada acesso à memória custaria o dobro

Tópico 7 · a cache das traduções

Síntese projetada

O problema — sem TLB, cada acesso à memória exigiria DOIS acessos: um à tabela de páginas e outro ao dado.

Aprofundamento

A tabela de páginas fica na memória principal. Sem nenhum apoio adicional, cada acesso do programa à memória exigiria dois acessos físicos: um para ler a entrada da tabela de páginas e obter o quadro, outro para ler o dado. Com tabelas multinível, seriam quatro ou cinco acessos por referência. Isso dobraria ou quintuplicaria o tempo de acesso à memória, tornando a paginação impraticável.

A TLB (Translation Lookaside Buffer) é a solução: uma pequena cache, dentro da MMU, que guarda as traduções (número de página → número de quadro) usadas mais recentemente. É uma memória associativa — a busca é feita em paralelo em todas as entradas, comparando o número da página com todas as chaves ao mesmo tempo, e leva um ciclo. Nos processadores atuais ela é organizada em dois níveis: o primeiro (L1 dTLB) tem 64 a 96 entradas para páginas de 4 KB e o segundo (STLB, compartilhado entre código e dados) de 1.000 a 3.000 — o que cobre, com páginas de 4 KB, apenas 4 a 12 MB de memória. É pouco diante dos working sets reais, e por isso páginas grandes (2 MB) são a solução padrão para cargas com muita memória: uma entrada de TLB passa a cobrir 512 vezes mais. Graças à localidade, mesmo esse alcance modesto atende a grande maioria dos acessos.

Num acerto (hit) na TLB, o número do quadro sai imediatamente e o único acesso à memória é o do dado. Numa falha (miss), a MMU (ou o SO, em arquiteturas como MIPS que tratam a TLB por software) consulta a tabela de páginas na memória, obtém a tradução, carrega-a na TLB — substituindo alguma entrada — e o acesso prossegue. O tempo efetivo de acesso é a média ponderada: com taxa de acerto h, tempo de acesso à memória t e busca na TLB ε, TEA = h·(ε + t) + (1 − h)·(ε + 2t). Com h = 98% e t = 100 ns, o tempo efetivo fica em torno de 102 ns — quase o de um único acesso.

A TLB traz um custo escondido na troca de contexto: as traduções que ela guarda pertencem ao processo corrente, e quando o SO troca a tabela de páginas, elas se tornam inválidas. Ou a TLB é esvaziada (flush) a cada troca — e o novo processo começa com uma TLB fria, sofrendo várias falhas até aquecê-la — ou cada entrada carrega um identificador de espaço de endereçamento (ASID) para que traduções de processos distintos coexistam. É mais um motivo pelo qual trocar entre threads do mesmo processo é mais barato que trocar entre processos: a TLB não precisa ser invalidada.

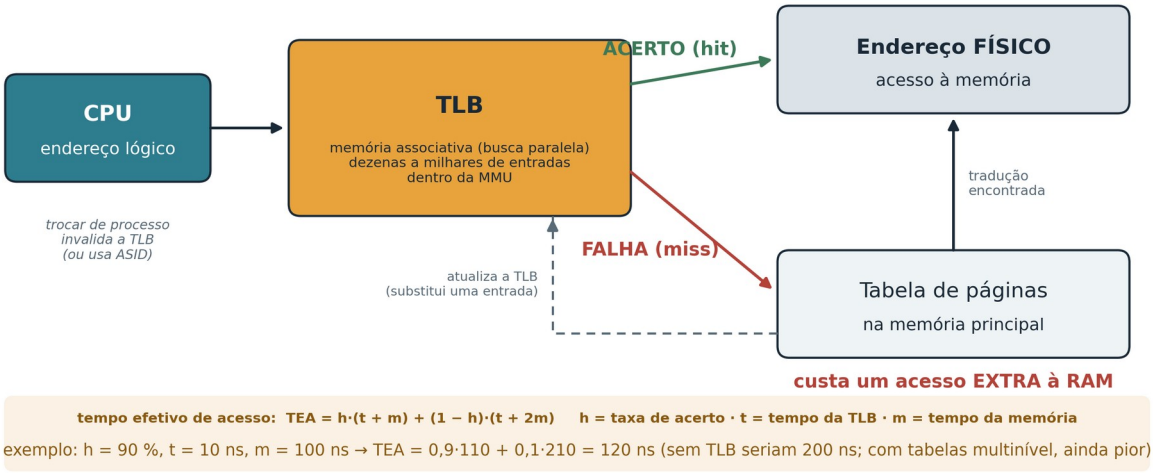


Figura 16 — Caminho da tradução com e sem acerto na TLB.

COMO LER A FIGURA

A CPU gera um endereço lógico que vai primeiro à TLB (caixa laranja). Dali saem dois caminhos. O verde, para cima, é o acerto: a tradução estava lá, e o endereço físico é formado em um ciclo. O vermelho, para baixo, é a falha: é preciso ir à tabela de páginas na memória principal, o que custa um acesso extra à RAM; a tradução encontrada é copiada para a TLB (seta 'atualiza a TLB') antes de o acesso prosseguir.

A frase de rodapé dá a conta que a prova pede: com 90 % de acerto, 10 ns de TLB e 100 ns de memória, o tempo efetivo é 0,9·110 + 0,1·210 = 120 ns — contra 200 ns sem TLB. Vale saber montar essa média ponderada.

Falta de página e políticas de substituição

Tópico 7 · o que o SO faz quando o bit de validade é zero

Síntese projetada

Sequência da falta — trap → o SO confere se a referência é válida (senão, encerra o processo) → há quadro livre? senão escolhe uma vítima → se ela está modificada (bit M), grava no disco → carrega a página → atualiza a tabela → reexecuta a instrução.

FIFO — substitui a mais antiga. Simples, mas sofre a anomalia de Belady: mais quadros podem dar MAIS faltas — cadeia 1-2-3-4-1-2-5-1-2-3-4-5: 3 quadros = 9 faltas, 4 quadros = 10 (foi a questão do ENADE 2017).

LRU — substitui a menos recentemente usada. Aproxima bem o ótimo, mas exige apoio de hardware (contador ou pilha atualizados a cada acesso).

Ótimo (OPT) — substitui a que será usada mais tarde. Exige conhecer o futuro: serve como REFERÊNCIA teórica, não é implementável.

NRU, segunda chance e clock — aproximações práticas: NRU classifica as páginas pelos bits R e M; segunda chance/clock é o FIFO que poupa a página com R = 1 (zera o bit e dá outra volta).

Aprofundamento

Quando a MMU encontra uma entrada com bit de validade zerado, gera a exceção de falta de página (page fault) e o controle passa ao SO. A sequência de tratamento tem etapas fixas que a prova pede em ordem. O SO verifica se a referência é legítima — se o endereço pertence ao espaço do processo; se não, é um erro de segmentação e o processo é encerrado. Se é legítima, localiza a página no disco (na área de swap ou no arquivo executável). Procura um quadro livre; se não houver, invoca o algoritmo de substituição para escolher uma vítima. Se a vítima

tem o bit M ligado, grava-a no disco antes — uma operação de E/S. Carrega a página desejada no quadro — outra operação de E/S. Atualiza a tabela de páginas (quadro, V = 1) e a TLB. Por fim, reexecuta a instrução que causou a falta, que agora encontra a página presente.

Como cada falta custa milissegundos, o algoritmo de substituição — a escolha da vítima — é decisivo. A métrica para compará-los é simular a mesma cadeia de referências (a sequência de números de página acessados) com o mesmo número de quadros e contar as faltas. Toda cadeia começa com faltas inevitáveis, as compulsórias, enquanto os quadros estão vazios; elas contam.

FIFO substitui a página que está há mais tempo na memória, independentemente de estar em uso. É trivial (uma fila) mas ignora a localidade: uma página muito usada, só por ter sido carregada cedo, é expulsa. E sofre da anomalia de Belady, descoberta em 1969: para certas cadeias, aumentar o número de quadros aumenta o número de faltas. Com a cadeia 1,2,3,4,1,2,5,1,2,3,4,5, FIFO produz 9 faltas com 3 quadros e 10 com 4. Comprar mais memória piorar o desempenho é contraintuitivo o bastante para ser questão de prova.

O algoritmo ótimo (OPT, ou MIN, de Belady) substitui a página que será referenciada mais tarde no futuro — ou nunca mais. Produz o mínimo possível de faltas para qualquer cadeia, mas exige conhecer o futuro, o que nenhum sistema real tem. Serve como limite inferior para avaliar os demais: um algoritmo é tão bom quanto mais se aproxima do OPT. Quando a questão pede 'o algoritmo ótimo', a resposta é sempre 'referência teórica, não implementável'.

LRU (Least Recently Used) substitui a página menos recentemente usada, apostando na localidade temporal: o passado recente é o melhor previsor do futuro próximo. Aproxima bem o OPT na maioria das cargas e, como o OPT, é um algoritmo de pilha — o conjunto de páginas residentes com n quadros é sempre subconjunto do conjunto com n + 1 —, o que o torna imune à anomalia de Belady. O problema é a implementação: manter a ordem exata de uso exige atualizar uma lista ou um contador a cada acesso à memória, o que é caro demais para fazer por software e raro em hardware.

Por isso os sistemas reais usam aproximações do LRU baseadas no bit de referência. NRU (Not Recently Used) classifica as páginas pelos bits R e M em quatro classes e escolhe uma da classe mais baixa (não referenciada, não modificada); o SO zera os bits R periodicamente. Segunda chance é o FIFO com uma ressalva: ao chegar à página mais antiga, se seu bit R estiver ligado, o algoritmo o zera e dá à página uma nova oportunidade, passando à seguinte. O algoritmo do relógio (clock) é a segunda chance com os quadros organizados em círculo e um ponteiro que gira — mesma lógica, implementação mais eficiente. O Linux usa uma variante com duas listas (ativa e inativa) que aproxima o LRU com custo constante.

Working set, thrashing e alocação de quadros

Tópico 7 · o diagnóstico que a prova adora cobrar

Síntese projetada

Local × global — na alocação LOCAL o processo só disputa os próprios quadros; na GLOBAL pode tomar quadros de outro — e sua taxa de faltas passa a depender do comportamento alheio.

Aprofundamento

O working set (conjunto de trabalho), definido por Peter Denning em 1968, é o conjunto de páginas que um processo referenciou nas últimas Δ referências — uma janela que desliza no tempo. Pela localidade, esse conjunto é muito menor que o espaço de endereçamento total e muda lentamente, com saltos quando o programa passa de uma fase para outra (do carregamento para o processamento, de uma estrutura de dados para outra). O working set é, na prática, a quantidade de memória de que o processo precisa agora para executar sem faltas excessivas.

O princípio que decorre disso é simples: enquanto a soma dos working sets de todos os processos residentes couber no número de quadros disponíveis, o sistema opera bem. Quando a soma ultrapassa a memória física, algum processo — sob alocação global, todos eles — fica com menos quadros do que seu working set exige. Ele passa a gerar uma falta de página a cada poucos acessos; cada falta expulsa uma página que outro processo está

usando, que por sua vez gera uma falta; e o sistema entra em thrashing: os processos passam mais tempo esperando o disco trazer páginas do que executando.

O quadro sintomático é reconhecível: a utilização da CPU despenca (todos os processos estão bloqueados esperando E/S de página), a fila do disco fica permanentemente cheia, e a taxa de faltas de página sobe ordens de grandeza. O sistema não travou — está trabalhando furiosamente — mas não produz nada. E aqui está a armadilha que a questão 9 do caderno explora: no modelo clássico, o escalonador de longo prazo, ao observar a CPU ociosa, conclui que há capacidade sobrando e admite mais processos, o que reduz ainda mais os quadros por processo e aprofunda o colapso. A intervenção correta é a contrária à intuição: reduzir o grau de multiprogramação, suspendendo processos para devolver quadros aos que ficam. Os sistemas de propósito geral atuais não têm escalonador de longo prazo — todo processo criado é admitido — e reagem à pressão de memória de outras formas: paginando para o swap, encolhendo o page cache, comprimindo páginas (zswap, compressão de memória no Windows e no macOS) e, no limite, matando um processo (o OOM killer do Linux). O princípio é o mesmo: reduzir a demanda por quadros.

Duas ferramentas ajudam a prevenir o thrashing. O modelo do working set faz o SO estimar o working set de cada processo (com o bit de referência e um relógio) e só admitir um novo processo se houver quadros para ele. A frequência de faltas de página (PFF) é mais direta: monitora a taxa de faltas de cada processo; se ela sobe acima de um limite, dá-lhe mais quadros; se cai abaixo de outro, tira-lhe alguns; se não há quadros para dar a quem precisa, suspende um processo.

A política de alocação define de onde vem o quadro da vítima. Na alocação local, o processo que sofreu a falta só pode substituir páginas suas — seu número de quadros é fixo, e seu desempenho depende só do seu comportamento, mas ele pode estar sofrendo faltas enquanto outro processo tem quadros sobrando. Na alocação global, a vítima pode ser de qualquer processo — o sistema se adapta melhor às necessidades variáveis, e é o que a maioria dos sistemas faz, mas um processo passa a poder ser prejudicado pelo comportamento dos outros, e o thrashing de um contamina todos. A alocação de quadros entre processos pode ser igualitária (o mesmo número para todos) ou proporcional ao tamanho ou à prioridade de cada um.

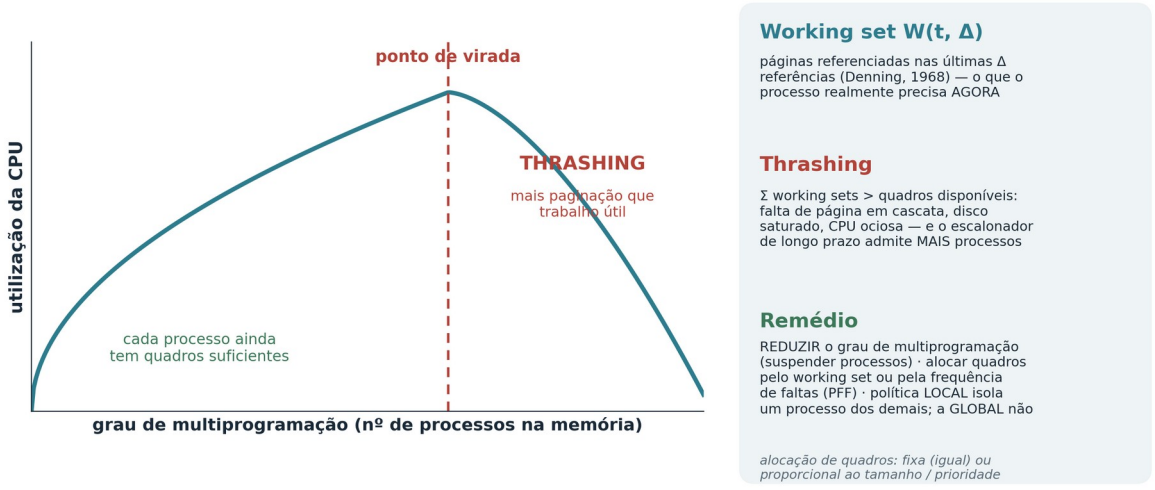


Figura 17 — Utilização da CPU em função do grau de multiprogramação.

COMO LER A FIGURA

O gráfico é o de Denning e aparece em todos os livros-texto. O eixo horizontal é o grau de multiprogramação — quantos processos estão residentes na memória; o vertical é a utilização da CPU. À esquerda do ponto de virada, adicionar processos aumenta a utilização: enquanto um espera E/S, outro executa, e cada um ainda tem quadros suficientes para seu working set. No ponto de virada, a soma dos working sets alcança a memória física.

À direita, cada processo adicional tira quadros dos demais; a taxa de faltas explode e a utilização da CPU cai a pique — a curva descendente é o thrashing. O painel à direita é o que se deve levar para a prova: a definição de working set, o diagnóstico (CPU baixa, disco saturado, faltas em alta — e o escalonador de longo prazo admitindo mais processos) e o remédio: reduzir o grau de multiprogramação, nunca aumentá-lo; a alocação local isola um processo dos demais.

Termos que a prova usa

VOCABULÁRIO DA SEÇÃO 4

Memória virtual — espaço lógico maior que o físico; partes sob demanda a partir do disco.

Página / quadro — bloco lógico / bloco físico de mesmo tamanho.

Tabela de páginas — mapeia página → quadro; uma por processo.

Deslocamento — posição dentro da página; não é traduzido.

Bits V, R, M — válida/presente, referenciada, modificada (dirty).

Tabela multinível / invertida — soluções para o tamanho da tabela.

Segmentação — divisão por unidades lógicas de tamanho variável.

TLB — cache associativa de traduções; acerto evita o acesso à tabela.

Tempo efetivo de acesso — média ponderada pelo acerto na TLB.

Falta de página — exceção quando V = 0; trap para o SO.

Faltas compulsórias — as inevitáveis com quadros vazios; contam.

FIFO / OPT / LRU — mais antiga / usada mais tarde / menos recentemente usada.

Anomalia de Belady — mais quadros, mais faltas — só em algoritmos não de pilha, como FIFO.

NRU, segunda chance, relógio — aproximações do LRU via bit R.

Working set — páginas usadas na janela recente Δ.

Thrashing — colapso por paginação excessiva quando os working sets não cabem.

Alocação local / global — vítima só do próprio processo / de qualquer processo.

Erros mais comuns

ONDE OS ESTUDANTES ERRAM

- Não contar as faltas compulsórias iniciais. Elas são faltas de página; o enunciado normalmente manda contá-las.
- Contar duas faltas por substituição (uma pela vítima, outra pela nova). É um único evento.
- Achar que FIFO e LRU dão sempre o mesmo resultado. Coincidem em algumas cadeias, não em geral.
- Traduzir o deslocamento. Só o número da página é traduzido.
- Propor aumentar o grau de multiprogramação para 'manter a CPU ocupada' durante o thrashing. É o que o agrava.
- Chamar o algoritmo ótimo de implementável. Ele exige conhecer o futuro.

Questões desta seção

Consulte o caderno de questões: questões 8 e 9. O gabarito comentado, com a análise de cada distrator, está no documento de gabarito.

Seção 5 — Sistemas de Arquivos e Entrada/Saída

Tópicos 8 e 9 — organização em disco, diretórios, subsistema de E/S e proteção

- T8 · Sistemas de arquivos: organização, diretórios e alocação em disco
- T9 · Gerência de E/S, drivers, shell e compartilhamento de recursos

A última seção reúne os dois tópicos do Objetivo de Aprendizagem 4: sistemas de arquivos (T8) e entrada e saída (T9). São os assuntos em que o SO mais se aproxima do que o usuário vê — arquivos, diretórios, permissões, impressoras — e por isso as questões costumam ter enunciados de projeto (nenhum dos dois tópicos apareceu nas provas de 2005 a 2021, mas ambos constam da Matriz de Referência): escolher um sistema de arquivos para um requisito, justificar um método de alocação, identificar a premissa errada numa configuração de E/S.

O eixo da seção é a distância entre a abstração e o hardware. Um arquivo é uma sequência de bytes com nome; por baixo, são blocos espalhados por um disco, apontados por um inode, listados num diretório, protegidos por permissões, e acessados por uma pilha de software que vai do read() ao controlador, passando por buffers, caches e drivers.

Arquivo, metadados e inode

Tópico 8 · o que o sistema guarda além do conteúdo, e como organiza os nomes

Síntese projetada

> **Métodos de acesso —** SEQUENCIAL (do início ao fim) · DIRETO/relativo (salta para o registro n) · INDEXADO (índice sobre uma chave).

Aprofundamento

Do ponto de vista do usuário, um arquivo é uma coleção nomeada de informação persistente. Do ponto de vista do SO, é uma sequência de bytes (no Unix; outros sistemas impuseram estruturas de registro) armazenada em blocos de disco, mais um conjunto de metadados que descrevem o arquivo: tipo, tamanho, proprietário, permissões, marcas de tempo, e a localização dos blocos. Separar dados de metadados é a primeira decisão de projeto de qualquer sistema de arquivos.

No Unix e seus descendentes, os metadados de cada arquivo ficam numa estrutura chamada inode (index node), identificada por um número. O inode guarda tudo sobre o arquivo — menos o nome. O nome fica na entrada de diretório, que é essencialmente um par (nome, número do inode). Essa separação é o que permite hard links: dois nomes, em diretórios diferentes, apontando para o mesmo inode e portanto para o mesmo arquivo. O inode mantém um contador de ligações, e o arquivo só é de fato removido quando o contador chega a zero — 'apagar' um arquivo é remover um nome, não os dados.

Os métodos de acesso definem como o conteúdo pode ser lido. No acesso sequencial, os registros são lidos em ordem, do início ao fim, e a posição corrente avança a cada leitura — é o modelo de fitas, de fluxos e da maioria dos programas comuns. No acesso direto (ou relativo), qualquer registro pode ser lido diretamente por sua posição, sem ler os anteriores — o que exige que os blocos possam ser localizados por cálculo ou por índice; bancos de dados dependem disso. No acesso indexado, um índice separado mapeia chaves em posições, permitindo buscar por conteúdo. O sistema de arquivos precisa suportar o acesso direto eficiente para que o acesso indexado seja viável — e é isso que orienta a escolha do método de alocação, mais adiante.

Os atributos de um arquivo — o que se vê com ls -l ou nas propriedades do Windows — são os metadados expostos: nome, tipo, tamanho, proprietário e grupo, permissões, marcas de tempo (no inode clássico do UNIX: atime, último acesso; mtime, última modificação do conteúdo; e ctime, última mudança dos metadados — não a criação, que só o ext4 e sistemas mais novos registram como crtime), e, dependendo do sistema, atributos estendidos e listas de controle de acesso. As operações sobre arquivos que o SO oferece como chamadas de sistema são criar, abrir, ler, escrever, posicionar (seek), fechar, remover, renomear e consultar/alterar atributos.

Abrir um arquivo cria uma entrada na tabela de arquivos abertos do processo e devolve um descritor, para que as operações seguintes não precisem repetir a busca pelo nome.

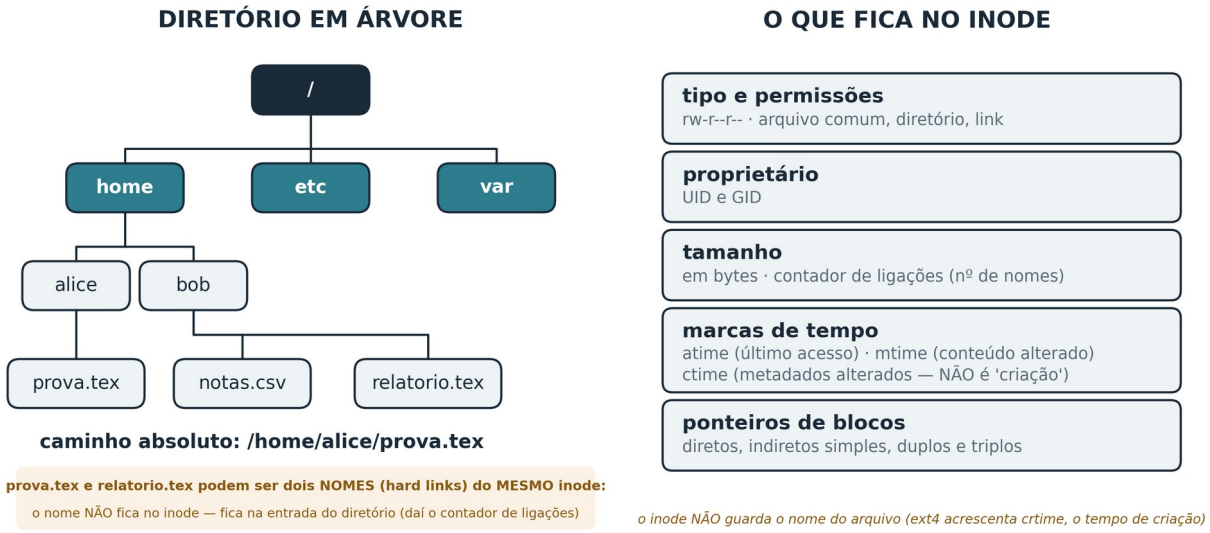


Figura 19 — Diretório em árvore e conteúdo do inode.

COMO LER A FIGURA

À esquerda, uma árvore de diretórios: a raiz no topo, três subdiretórios de primeiro nível, e um arquivo de alice em /home/alice e dois de bob em /home/bob. O caminho absoluto de prova.tex é a sequência de nomes da raiz até ele. Cada retângulo de diretório é, internamente, uma lista de pares (nome, número de inode).

À direita, o que o inode de um arquivo contém. Percorra a lista e note o que não está lá: o nome. A frase do rodapé existe para gravar isso — o nome fica no diretório, e é por isso que um mesmo inode pode ter vários nomes (hard links) e que renomear um arquivo é uma operação sobre o diretório, não sobre o arquivo.

Alocação de arquivos em disco

Tópico 8 · três esquemas e seus compromissos

Síntese projetada

O critério de escolha — padrão de acesso predominante: sequencial favorece a encadeada; aleatório exige a indexada.

Aprofundamento

O disco é dividido em blocos (tipicamente 4 KB, alinhados aos setores), e o sistema de arquivos precisa decidir em que blocos gravar cada arquivo e como localizá-los depois. Os três esquemas clássicos espelham, um a um, a discussão de alocação de memória — e a prova frequentemente pede que o estudante associe cada esquema às suas características de acesso, fragmentação e crescimento.

Na alocação contígua, o arquivo ocupa uma sequência de blocos consecutivos; a entrada de diretório guarda apenas o bloco inicial e o tamanho. É o melhor esquema para leitura sequencial (o cabeçote do disco quase não se move) e permite acesso direto trivial — o bloco lógico i está no bloco físico início + i. Mas sofre fragmentação externa exatamente como as partições variáveis de memória, e tem dificuldade com o crescimento: se o bloco seguinte estiver ocupado, o arquivo precisa ser movido inteiro. Foi o esquema dos primeiros sistemas e sobrevive em mídias de gravação única (CD, DVD).

Na alocação encadeada, cada bloco do arquivo contém, além dos dados, um ponteiro para o bloco seguinte; a entrada de diretório guarda o primeiro e o último. Não há fragmentação externa — qualquer bloco livre serve —

e o arquivo cresce sem limite. Mas o acesso direto é péssimo: para chegar ao bloco i é preciso ler os i − 1 anteriores, seguindo a cadeia. Cada bloco perde alguns bytes para o ponteiro, e um ponteiro corrompido perde o resto do arquivo. A variante que resolve o acesso direto é a FAT (File Allocation Table): os ponteiros saem dos blocos e vão para uma tabela única no início do volume, com uma entrada por bloco; seguir a cadeia passa a ser percorrer a tabela, que pode ficar em cache na memória. É o esquema do FAT12/16/32 e do exFAT.

Na alocação indexada, todos os ponteiros para os blocos de um arquivo são reunidos em um bloco de índice; a entrada de diretório aponta para ele. O acesso direto é eficiente — o bloco i está na posição i do índice — e não há fragmentação externa. O custo é o espaço do índice: um arquivo de um único bloco gasta dois. E há um limite de tamanho — um bloco de 4 KB com ponteiros de 4 bytes indexa 1024 blocos, 4 MB. Para arquivos maiores, o índice é encadeado, ou tem vários níveis, ou — o esquema do Unix — combina os dois: o inode contém alguns ponteiros diretos (para os primeiros blocos, que resolvem arquivos pequenos sem indireção), um ponteiro indireto simples (para um bloco cheio de ponteiros), um indireto duplo e um indireto triplo. O ext4 substituiu esse esquema por extents — pares (bloco inicial, comprimento) que descrevem faixas contíguas — justamente para reaproveitar a eficiência sequencial da alocação contígua em arquivos grandes.

A regra de decisão que a prova espera: predomina acesso sequencial e crescimento incremental (logs, fluxos) → encadeada; predomina acesso aleatório a posições arbitrárias (bancos de dados, imagens de grande porte lidas por regiões) → indexada; leitura sequencial de arquivos que não mudam → contígua.

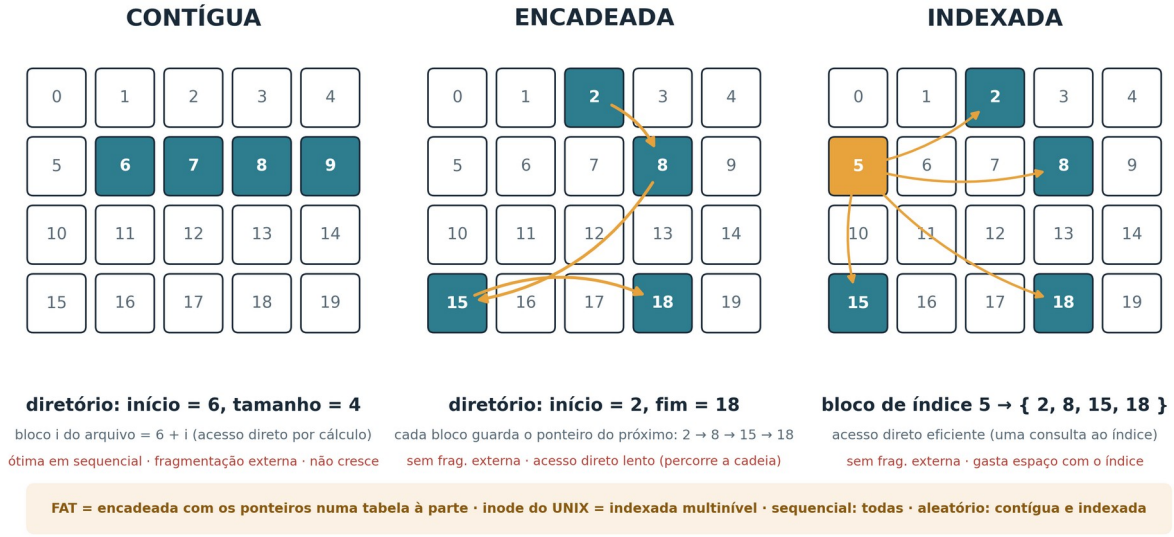


Figura 18 — Alocação contígua, encadeada e indexada.

COMO LER A FIGURA

As três grades representam o mesmo disco de 20 blocos, e os blocos em destaque são o mesmo arquivo de quatro blocos em cada esquema. Na contígua, os blocos 6 a 9 são consecutivos e a entrada de diretório só precisa de 'início = 6, tamanho = 4'. Na encadeada, o arquivo está nos blocos 2, 8, 15 e 18, e as setas laranja são os ponteiros que cada bloco guarda para o seguinte — para chegar ao 18 é preciso passar pelo 2, pelo 8 e pelo 15. Na indexada, os mesmos quatro blocos são apontados a partir do bloco laranja (5), o bloco de índice, e qualquer um deles pode ser alcançado em um passo.

As três linhas abaixo de cada grade resumem o trade-off, e a frase de rodapé dá a regra prática. Ao responder uma questão de associação, procure as palavras-chave: 'ponteiro em cada bloco' é encadeada; 'blocos sequenciais' ou 'cálculo' é contígua; 'bloco de índice' ou 'tabela de ponteiros' é indexada.

Espaços livres, journaling e sistemas de arquivos

Tópico 8 · integridade depois da queda de energia

Síntese projetada

Gerência de espaços livres — MAPA DE BITS (um bit por bloco — fácil achar sequências contíguas) × LISTA ENCADEADA (economiza espaço, mas não localiza contiguidade).

Journaling — grava as operações num diário ANTES de aplicá-las. Após falha, basta reaplicar ou descartar transações incompletas — dispensa varrer o volume inteiro.

ext4 e XFS — Linux, com journaling; XFS se destaca em arquivos muito grandes.

NTFS — Windows, com journaling, permissões por ACL e compressão.

FAT32 — sem journaling e com limite de 4 GB por arquivo. Sobrevive pela compatibilidade universal.

Aprofundamento

Além de saber onde estão os blocos de cada arquivo, o sistema precisa saber quais blocos estão livres. O mapa de bits (bitmap) dedica um bit a cada bloco do volume: 1 livre, 0 ocupado (ou o inverso). É compacto — um volume de 1 TB com blocos de 4 KB precisa de 32 MB de bitmap — e permite encontrar sequências de blocos livres contíguos com operações simples sobre palavras de bits, o que é valioso para reduzir a fragmentação de arquivos grandes. A lista encadeada de blocos livres guarda um ponteiro no primeiro bloco livre para o seguinte, e assim por diante; não gasta espaço extra (usa os próprios blocos livres), mas percorrê-la é lento e não há como localizar contiguidade. Variantes agrupam ponteiros em blocos (grouping) ou guardam pares (início, contagem) de faixas livres (counting).

O problema que o journaling resolve é a inconsistência dos metadados após uma interrupção abrupta. Operações triviais para o usuário envolvem várias escritas em estruturas distintas: criar um arquivo altera o bitmap de inodes, o inode novo, a entrada de diretório e possivelmente o bitmap de blocos. Se a energia cair entre duas dessas escritas, o volume fica num estado intermediário — um inode alocado sem nome, um bloco marcado como usado que ninguém referencia. Os sistemas antigos resolviam isso com uma varredura completa na inicialização (fsck, chkdsk), que em volumes de terabytes leva horas.

Com journaling, o sistema de arquivos funciona como um banco de dados transacional: antes de aplicar as escritas às estruturas definitivas, grava a descrição da operação num diário (journal), uma área reservada e sequencial; só depois de o registro estar completo no diário, aplica as mudanças; e só então marca a transação como concluída. Se houver falha, a recuperação lê apenas o diário: transações com registro completo são reaplicadas (redo), transações incompletas são descartadas. O volume volta a um estado consistente em segundos, independentemente do tamanho.

O ext4, segundo a documentação do kernel Linux, oferece três modos. No data=ordered, o padrão, só os metadados vão para o diário, mas o sistema garante que os blocos de dados sejam escritos antes de os metadados que os referenciam serem confirmados — evita que um arquivo aponte para lixo. No data=journal, dados e metadados passam pelo diário: é o mais seguro e o mais lento, porque tudo é escrito duas vezes. No data=writeback, só os metadados são diarizados e sem ordenação com os dados: é o mais rápido, mas um arquivo escrito logo antes da queda pode conter dados antigos. A distinção entre journaling de metadados e journaling completo é o que a prova costuma cobrar.

Os sistemas de arquivos que se deve conhecer pelo nome. ext4 é o padrão do Linux: journaling, extents, arquivos de até 16 TB, volumes de até 1 EB. XFS, também Linux, com journaling e projeto voltado a arquivos e volumes muito grandes e a paralelismo; é o padrão do Red Hat. Btrfs e ZFS vão além do journaling com copy-on-write, snapshots e checksums de dados. NTFS é o do Windows: journaling de metadados, permissões por listas de controle de acesso (ACL), compressão e criptografia nativas. FAT32 não tem journaling nem permissões, e limita arquivos a 4 GB — mas é lido por praticamente qualquer dispositivo, e por isso sobrevive em pendrives, cartões de memória e na partição EFI. exFAT remove o limite de 4 GB mantendo a simplicidade. Quando a questão fala em integridade após queda de energia, a palavra-chave é journaling; quando fala em arquivos acima de 4 GB, o FAT32 está eliminado.

E/S programada, por interrupção e por DMA

Tópico 9 · quem faz o trabalho de mover os dados

Síntese projetada

A pegadinha — polling NÃO libera a CPU — é espera ocupada. Quem libera a CPU e transfere o bloco inteiro é o DMA.

Aprofundamento

Há três técnicas para a CPU realizar uma operação de E/S, e a distinção entre elas é uma das questões mais recorrentes do tópico — inclusive a de afirmativa negativa do caderno. O critério é: quem faz o trabalho de mover os dados, e o que a CPU faz enquanto isso.

Na E/S programada (polling), a CPU faz tudo: escreve o comando no registrador do controlador e depois fica em um laço lendo o registrador de estado até que o controlador indique 'pronto'; então lê o dado do registrador de dados, copia-o para a memória, e repete para o próximo byte ou palavra. Isso é espera ocupada (busy waiting): a CPU está 100% ocupada mas não executa nada útil, e nenhum outro processo roda. É simples, previsível e adequada só para dispositivos muito rápidos (em que a espera é curta) ou para sistemas embarcados sem multiprogramação. A afirmação 'a E/S programada libera a CPU' é exatamente o oposto da realidade.

Na E/S por interrupção, a CPU escreve o comando e vai fazer outra coisa — escalona outro processo. Quando o controlador tem um dado pronto, gera uma interrupção; o tratador copia o dado para a memória, prepara o próximo e retorna. A CPU só é tomada pelo tempo do tratador, e os intervalos de espera do dispositivo ficam disponíveis para outros processos. No modelo clássico o custo é uma interrupção por byte ou palavra — para um disco a centenas de megabytes por segundo, seriam dezenas de milhões de interrupções por segundo, inviável. Controladores modernos amortecem isso com FIFOs internas (interrompem quando o buffer enche) e os sistemas praticam coalescência de interrupções e polling adaptativo (o NAPI da pilha de rede do Linux alterna entre interrupção e polling conforme a carga). Mas o argumento de fundo permanece: para transferências grandes, quem move os dados tem de ser o DMA.

O DMA (Direct Memory Access) resolve isso com um controlador dedicado que transfere dados entre o dispositivo e a memória sem passar pela CPU. A CPU programa o controlador de DMA com o endereço de memória, a quantidade de bytes e a direção, e vai embora; o DMA arbitra o barramento com a CPU, move o bloco inteiro e, só ao final, gera uma única interrupção. Para um bloco de 4 KB, uma interrupção em vez de quatro mil. Todo disco, SSD, placa de rede e GPU moderno usa DMA; é a técnica que torna possível E/S de alta taxa sem paralisar o sistema. Seu único custo para a CPU é a disputa pelo barramento de memória (cycle stealing) enquanto a transferência ocorre.

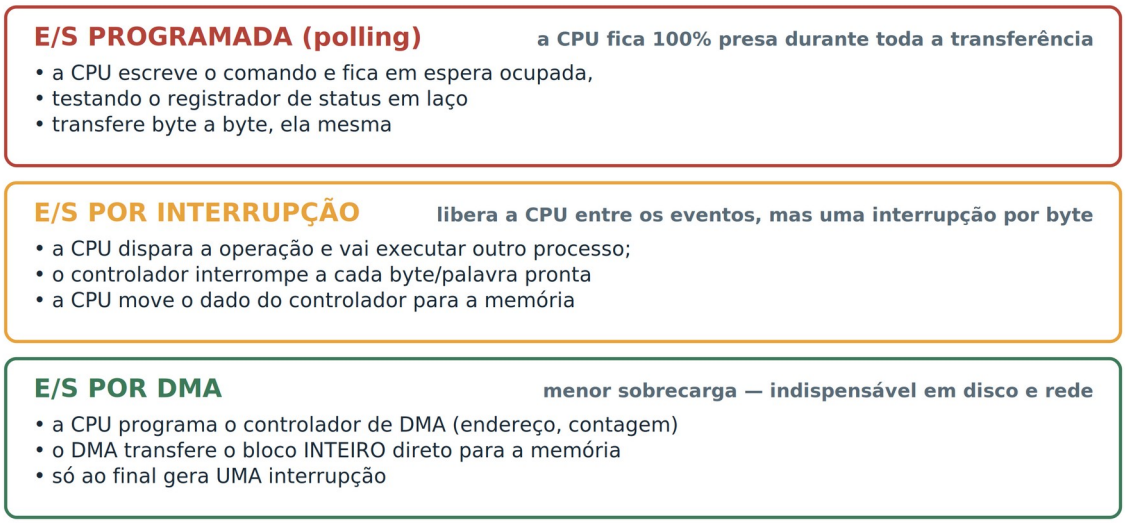


Figura 22 — E/S programada, por interrupção e por DMA.

COMO LER A FIGURA

Cada caixa é uma técnica; a cor da borda vai do vermelho (pior para a CPU) ao verde (melhor). Dentro de cada uma, os três itens descrevem quem faz o quê. À direita do título, a frase curta é a consequência para a CPU — e é ela que a questão de afirmativa negativa testa: 'presa 100%' para o polling, 'liberada entre eventos, mas uma interrupção por byte' para a E/S por interrupção, 'menor sobrecarga' para o DMA.

A frase de rodapé lembra que buffering, caching e spooling são técnicas de software que se aplicam sobre qualquer uma das três — não são uma quarta técnica de transferência.

Termos que a prova usa

VOCABULÁRIO DA SEÇÃO 5

Arquivo — coleção nomeada de bytes persistentes, com metadados.

Inode — estrutura de metadados do arquivo; não contém o nome.

Entrada de diretório — par (nome, número de inode).

Hard link / link simbólico — segundo nome para o mesmo inode / arquivo que contém um caminho.

Acesso sequencial / direto / indexado — em ordem / por posição / por chave.

Alocação contígua / encadeada / indexada — blocos seguidos / ponteiro em cada bloco / bloco de índice.

FAT — alocação encadeada com os ponteiros numa tabela central.

Extents — faixas (início, comprimento) de blocos contíguos — ext4, XFS, NTFS.

Mapa de bits / lista encadeada de livres — um bit por bloco / ponteiros nos blocos livres.

Journaling — diário de transações que garante consistência após falha.

data=ordered / journal / writeback — os três modos do ext4.

Driver — software que conhece o controlador de um dispositivo específico.

Dispositivo de bloco / de caractere — endereçável em blocos / fluxo de bytes.

Polling, interrupção, DMA — as três técnicas de E/S.

Buffering / caching / spooling — intermediação de velocidade / cópia para reuso / fila em disco.

rwx para dono, grupo, outros — permissões Unix; em diretório, x = atravessar.

FCFS, SSTF, SCAN, C-SCAN, LOOK — algoritmos de escalonamento de disco.

Erros mais comuns

ONDE OS ESTUDANTES ERRAM

- Colocar o nome do arquivo no inode. O nome está na entrada de diretório.
- Dizer que a alocação encadeada tem bom acesso direto. É preciso percorrer a cadeia.
- Atribuir journaling ao FAT32, ou esquecer seu limite de 4 GB por arquivo.
- Afirmar que a E/S programada libera a CPU. É espera ocupada — a CPU fica presa.
- Confundir buffer com cache. Buffer é a cópia em trânsito; cache é a cópia extra para reuso.
- Tratar o bit x de um diretório como 'executar o diretório'. Ele controla a travessia.
- Achar que SSTF não tem inanição. Tem, como o SJF; o SCAN é a solução.

Questões desta seção

Consulte o caderno de questões: questões 10, 11 e 12. O gabarito comentado, com a análise de cada distrator, está no documento de gabarito.

Apêndice — Conteúdo complementar

POR QUE ESTE CONTEÚDO NÃO ENTROU NA APRESENTAÇÃO

Os temas a seguir constam da Matriz de Referência e foram estudados pela squad. Inicialização do sistema (boot), estruturas de diretório e escalonamento de disco ficaram fora da apresentação por restrição de tempo, com um critério objetivo: nenhum deles registrou ocorrência nas seis edições do ENADE de Computação analisadas (2005, 2008, 2011, 2014, 2017 e 2021). Camadas do subsistema de E/S e buffering, caching, spooling e permissões SÃO apresentados, resumidos num único slide da Seção 5; estão aqui porque o aprofundamento não caberia na fala.

Inicialização do sistema (boot)

Tópico 3 · a cadeia genérica — cada elo carrega o seguinte

Aprofundamento

O processo de inicialização (boot, de bootstrap) resolve um paradoxo: o SO é quem carrega programas na memória, mas quem carrega o SO? A resposta é uma cadeia de programas cada vez maiores, em que cada elo carrega o seguinte. Conhecer a ordem dessa cadeia é o que permite diagnosticar uma falha de boot pela mensagem que aparece na tela — e é exatamente isso que a questão de ordenação cobra.

O primeiro elo é o firmware gravado na placa-mãe: o BIOS nos sistemas antigos, a UEFI nos atuais. Ao receber energia, o processador começa a executar em um endereço fixo que aponta para o firmware. Ele executa o POST (Power-On Self-Test), que testa e inventaria memória, processador e periféricos essenciais — é a mensagem de contagem de memória que se vê na tela — e configura o hardware básico. Até aqui nenhuma unidade de armazenamento foi lida.

Em seguida o firmware percorre a ordem de inicialização configurada (disco, USB, rede) à procura de um dispositivo inicializável. No esquema BIOS, ele lê o primeiro setor do disco, o MBR (Master Boot Record, 512 bytes), que contém um carregador mínimo e a tabela de partições; só cabe ali o primeiro estágio do GRUB. No esquema UEFI, ele lê a tabela de partições GPT, monta a partição de sistema EFI (formatada em FAT) e executa um arquivo .efi — o GRUB, o systemd-boot ou o próprio kernel com EFI stub. O esquema BIOS/MBR é legado; máquinas fabricadas desde meados da década de 2010 usam UEFI/GPT. A mensagem 'nenhum dispositivo inicializável encontrado' pertence a esta etapa: o firmware terminou o POST, mas não achou carregador válido em nenhum dispositivo.

O gerenciador de boot é o elo intermediário — GRUB 2 no Linux, Windows Boot Manager no Windows, systemd-boot em instalações mais enxutas. Ele lê seu arquivo de configuração, exibe o menu, permite escolher o kernel e editar parâmetros, carrega a imagem comprimida do kernel (vmlinuz) e o initramfs para a memória e transfere o controle ao kernel. Uma configuração corrompida do GRUB, um kernel apagado ou um initramfs incompatível produzem falhas nesta etapa — 'file not found', 'error: no such partition', ou o prompt de emergência do GRUB.

O kernel então descompacta-se, entra no modo de operação definitivo do processador (modo protegido ou longo no x86), inicializa as estruturas internas — tabelas de páginas, tratamento de interrupções, escalonador, detecção de CPUs adicionais — e executa o /init do initramfs. O initramfs é um sistema de arquivos temporário em RAM com o mínimo necessário para achar e montar a raiz real: módulos de driver de disco e de controladora, ferramentas para montar RAID e LVM, para desbloquear volumes criptografados. Concluída a montagem da raiz, o initramfs executa switch_root, que troca a raiz temporária pela definitiva e libera a memória.

Por fim o kernel executa o primeiro processo de modo usuário, com identificador 1 — o init, que nas distribuições Linux atuais é o systemd e no Windows corresponde ao conjunto smss.exe / wininit.exe / services.exe. Ele lê as unidades de serviço, resolve as dependências e inicia os serviços em paralelo até atingir o alvo padrão (multi-user.target para servidores, graphical.target para desktops). Todo processo do sistema descende do PID 1; quando ele termina, o sistema para. Falhas nesta etapa aparecem como serviços que não

sobem ou como o sistema caindo no modo de emergência — e podem ser investigadas com journalctl e systemd-analyze.

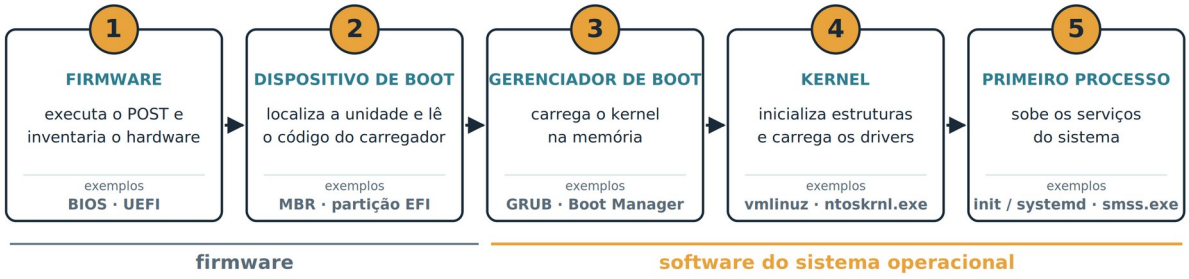


Figura 4 — Sequência de inicialização do sistema.

COMO LER A FIGURA

Os cinco cartões nomeiam a FUNÇÃO de cada elo da cadeia, não o produto que a cumpre — é assim que o ENADE cobra o tema. Sob cada cartão, em cinza, estão apenas exemplos de implementações que exercem aquela função em Linux e em Windows. Se a questão trocar GRUB por Windows Boot Manager, a posição na sequência não muda.

Os cinco cartões seguem a ordem temporal da esquerda para a direita. A barra cinza sob os dois primeiros marca o que é firmware — código gravado na placa-mãe, que existe antes de qualquer sistema operacional. A barra laranja sob os três últimos marca o que é software do sistema operacional, gravado no disco.

O ponto pedagógico central é a frase de rodapé: o modo kernel só começa a valer a partir do passo 3, quando há um núcleo em execução. Antes disso, o processador está em um estado inicial sem proteção — o que, aliás, é a razão de o firmware ser um alvo atraente para ataques e de a UEFI ter introduzido o Secure Boot, que verifica a assinatura digital de cada elo da cadeia antes de executá-lo.

Estruturas de diretório

Tópico 8 · da lista única ao grafo acíclico

Síntese

Nível único — uma lista para todo o sistema. Nomes precisam ser globalmente únicos — inviável com vários usuários.

Dois níveis — um diretório por usuário. Resolve a colisão de nomes, mas impede agrupamento lógico.

Árvore — diretórios aninhados sem limite. É o modelo usual, com caminhos absolutos e relativos.

Grafo acíclico — permite compartilhar um mesmo arquivo em vários diretórios por meio de ligações (links). Exige contador de referências para saber quando remover de fato.

Aprofundamento

O diretório é a estrutura que dá nomes aos arquivos e os organiza. Sua evolução histórica é uma sequência de problemas e soluções que a prova costuma percorrer.

O diretório de nível único é uma lista única para todo o sistema: todo arquivo tem um nome, e todos os nomes têm de ser distintos. Funciona para um usuário com poucos arquivos; com vários usuários, as colisões de nome tornam-se inevitáveis (dois 'relatorio.txt'), e não há como agrupar arquivos relacionados. O diretório de dois níveis dá a cada usuário seu próprio diretório: os nomes só precisam ser únicos dentro de cada um, e o

isolamento entre usuários é natural. Mas continua impossível agrupar arquivos de um mesmo projeto dentro do diretório do usuário, e compartilhar um arquivo entre usuários exige nomear o caminho completo.

O diretório em árvore generaliza: um diretório pode conter outros diretórios, sem limite de profundidade. Cada arquivo é identificado por um caminho — absoluto, da raiz até ele, ou relativo ao diretório corrente do processo. É o modelo de todos os sistemas modernos, e as operações sobre diretórios (criar, remover, listar, percorrer) são chamadas de sistema próprias. A remoção de um diretório não vazio é a operação delicada: ou se exige que esteja vazio, ou se remove recursivamente todo o conteúdo.

O grafo acíclico dirigido relaxa a árvore para permitir que um mesmo arquivo ou subdiretório apareça em mais de um lugar — compartilhamento. Duas implementações convivem. O hard link é uma segunda entrada de diretório apontando para o mesmo inode; é indistinguível da 'original', e o arquivo persiste enquanto restar um link — daí o contador de ligações no inode. O link simbólico (soft link) é um arquivo especial cujo conteúdo é um caminho; seguir o link é resolver esse caminho. Ele pode apontar para diretórios e para outros sistemas de arquivos, mas quebra se o alvo for removido (dangling link). Ciclos são proibidos ou tratados com cuidado porque tornariam a travessia recursiva infinita e a contagem de referências insuficiente para detectar arquivos inalcançáveis.

Camadas do subsistema de E/S

Tópico 9 · por que o mesmo read() serve para disco, teclado e rede

Síntese

Função do driver — é o único componente que conhece os detalhes do dispositivo; acima dele, tudo é genérico.

Aprofundamento

O subsistema de entrada e saída é a parte do SO que mais varia entre máquinas, porque há centenas de tipos de dispositivo, cada um com seus registradores, comandos e temporizações. A estratégia para domar essa variedade é a mesma da máquina de níveis: camadas, em que só a mais baixa conhece o dispositivo. Tanenbaum descreve quatro camadas de software sobre o hardware, e a prova pede que se saiba o que cada uma faz.

Na camada mais alta, o software de E/S no nível do usuário: as bibliotecas (a stdio do C, com seu buffer de printf) e o spooling, que veremos adiante. Abaixo, o software de E/S independente de dispositivo, no núcleo: é aqui que se dá nome aos dispositivos (o /dev/sda do Unix), se verificam permissões, se faz a bufferização genérica, se aloca e libera dispositivos dedicados, se converte tamanhos de bloco e se relatam erros de forma uniforme. É essa camada que faz um mesmo read() servir para disco, teclado e rede — o programa chama uma interface genérica, e ela decide para qual driver encaminhar.

Abaixo dela, os drivers de dispositivo: um por tipo (ou família) de dispositivo, escrito por quem conhece seus registradores. O driver recebe pedidos abstratos ('leia o bloco 4711') e os converte em sequências de comandos ao controlador; é o único componente que sabe que aquele disco específico precisa de determinado bit em determinado registrador. Como rodam em modo kernel (nos núcleos monolíticos), drivers defeituosos são a principal causa de falhas de sistema — o argumento central dos micronúcleos, que os expulsam para o espaço de usuário.

Na base do software, os tratadores de interrupção. Quando o controlador termina uma operação, gera uma interrupção; o tratador correspondente é executado, registra o resultado e acorda o driver, que estava bloqueado esperando. É a ponte entre o mundo assíncrono do hardware e o mundo sequencial do driver. Por baixo de tudo, o hardware: controladores (a eletrônica que fala com o dispositivo, com seus registradores de comando, estado e dados) e os dispositivos propriamente ditos. Dispositivos de bloco (disco, SSD) transferem em blocos endereçáveis; dispositivos de caractere (teclado, porta serial, impressora) transferem fluxos de bytes sem endereçamento.



Figura 20 — Camadas do subsistema de entrada e saída.

COMO LER A FIGURA

As cinco faixas são as camadas, do processo de usuário no topo ao hardware na base. A seta da esquerda mostra o pedido descendo — o read() do programa vira uma chamada genérica, que vira um comando de driver, que vira operação no controlador. A seta da direita mostra a resposta subindo — o controlador interrompe, o tratador acorda o driver, o driver devolve à camada genérica, que devolve ao processo.

A camada em laranja (drivers) é a única que conhece o hardware específico; a em azul (tratadores) é onde a interrupção é atendida. A frase de rodapé é o resultado: cada camada esconde detalhes da de baixo, e por isso o mesmo read() serve para qualquer dispositivo.

Buffering, caching, spooling e permissões

Tópico 9 · técnicas que amortecem a lentidão dos dispositivos

Síntese

Buffering — área intermediária que absorve a diferença de velocidade entre produtor e consumidor.

Caching — guarda em memória blocos recentemente acessados, aproveitando a localidade de referência.

Spooling — fila em disco que transforma um dispositivo não compartilhável (impressora) em recurso concorrente.

Permissões em Linux — rwx para dono, grupo e outros, verificadas pelo núcleo. Em diretório: x = atravessar, r = listar o conteúdo.

Aprofundamento

Três técnicas de software, todas na camada independente de dispositivo, tratam da diferença de velocidade e de comportamento entre a CPU e os dispositivos. Elas são parecidas o bastante para serem confundidas em prova, e vale ter a definição precisa de cada uma.

Buffering (uso de buffers, ou memória intermediária) resolve a incompatibilidade de velocidade ou de tamanho de transferência entre produtor e consumidor. O teclado produz um caractere por vez, em ritmo humano; o programa quer ler uma linha inteira: o buffer acumula. O disco entrega blocos de 4 KB; o programa pede 100 bytes: o buffer guarda o bloco e serve os pedidos seguintes sem novo acesso. O buffer duplo permite que o dispositivo preencha um enquanto o programa consome o outro, sobrepondo E/S e processamento. O buffer é transitório — existe para a transferência em curso.

Caching (uso de cache) guarda cópias de dados já lidos para servir pedidos repetidos sem ir ao dispositivo. O buffer cache (ou page cache) do SO mantém em RAM os blocos de disco recentemente acessados; graças à localidade, uma fração grande das leituras é atendida da memória, e as escritas podem ser adiadas e agrupadas (write-back). A diferença conceitual em relação ao buffer: o buffer guarda a única cópia de um dado em trânsito; a cache guarda uma cópia adicional, para acelerar reuso. Na prática, o Linux unifica os dois no page cache.

Spooling (Simultaneous Peripheral Operation On-Line) trata de dispositivos que não podem ser compartilhados de forma intercalada — a impressora é o exemplo clássico: se dois processos escrevessem nela ao mesmo tempo, as páginas sairiam misturadas. A solução é interpor um diretório em disco: cada processo 'imprime' gravando um arquivo de spool, e um processo servidor (o daemon de impressão) envia os arquivos à impressora, um de cada vez. Para os processos, a impressora parece compartilhável e instantânea; a serialização acontece no spool. A mesma ideia se aplica a filas de e-mail e de trabalhos em lote.

Por fim, as permissões — o mecanismo de proteção do sistema de arquivos. No modelo Unix, cada arquivo tem um proprietário e um grupo, e três conjuntos de três bits: leitura (r), escrita (w) e execução (x) para o proprietário, para o grupo e para os demais usuários (rwxr-xr-- é 754 em octal). O núcleo verifica essas permissões na abertura do arquivo, e a verificação é feita uma vez — por isso um arquivo aberto continua acessível mesmo que as permissões mudem depois. Em diretórios, o significado muda: r permite listar o conteúdo, w permite criar e remover entradas, e x permite atravessar o diretório — acessar caminhos abaixo dele. Um diretório com x mas sem r permite abrir um arquivo cujo nome se conhece, mas não descobrir que arquivos existem. Sistemas como NTFS e o ext4 com ACLs generalizam o modelo para listas arbitrárias de usuários e direitos.

Escalonamento de disco

Tópico 9 · em que ordem atender a fila de requisições

Síntese

SSTF causa inanição — requisições distantes do cabeçote podem esperar indefinidamente; o SCAN garante um limite de espera.

Aprofundamento

Em discos magnéticos, o tempo de uma operação é dominado pelo movimento mecânico: o tempo de busca (seek) para posicionar o cabeçote no cilindro certo e a latência rotacional para o setor passar sob ele — milissegundos, contra nanossegundos da RAM. Quando há várias requisições pendentes, a ordem em que são atendidas muda drasticamente o deslocamento total do cabeçote, e o escalonamento de disco é a política que decide essa ordem. O exemplo da figura é o de Silberschatz: fila 98, 183, 37, 122, 14, 124, 65, 67, cabeçote em 53.

FCFS atende na ordem de chegada. É justo e simples, mas o cabeçote vai e volta pelo disco inteiro: 640 cilindros percorridos no exemplo. SSTF (Shortest Seek Time First) atende sempre a requisição mais próxima da posição atual — é o SJF do disco, e reduz o total para 236 cilindros. Como o SJF, pode causar inanição: uma requisição distante pode esperar indefinidamente enquanto chegam requisições próximas. SCAN, o algoritmo do elevador, move o cabeçote numa direção atendendo tudo no caminho até a borda, e então inverte: também 236 no exemplo (indo primeiro em direção ao cilindro 0), mas com garantia de que nenhuma requisição espera mais que duas varreduras. C-SCAN (circular) só atende num sentido e retorna ao início sem atender, dando espera mais uniforme; LOOK e C-LOOK são as variantes que invertem na última requisição em vez de ir até a borda física.

Em SSDs não há cabeçote e o tempo de acesso é praticamente independente da posição, o que torna o escalonamento por posição irrelevante; o que importa passa a ser agrupar escritas e distribuir o desgaste. Os escalonadores de E/S modernos do Linux (mq-deadline, BFQ, Kyber) são projetados em torno de filas por CPU e de garantias de latência, não de minimização de seek — mas os algoritmos clássicos continuam sendo o que a prova cobra.

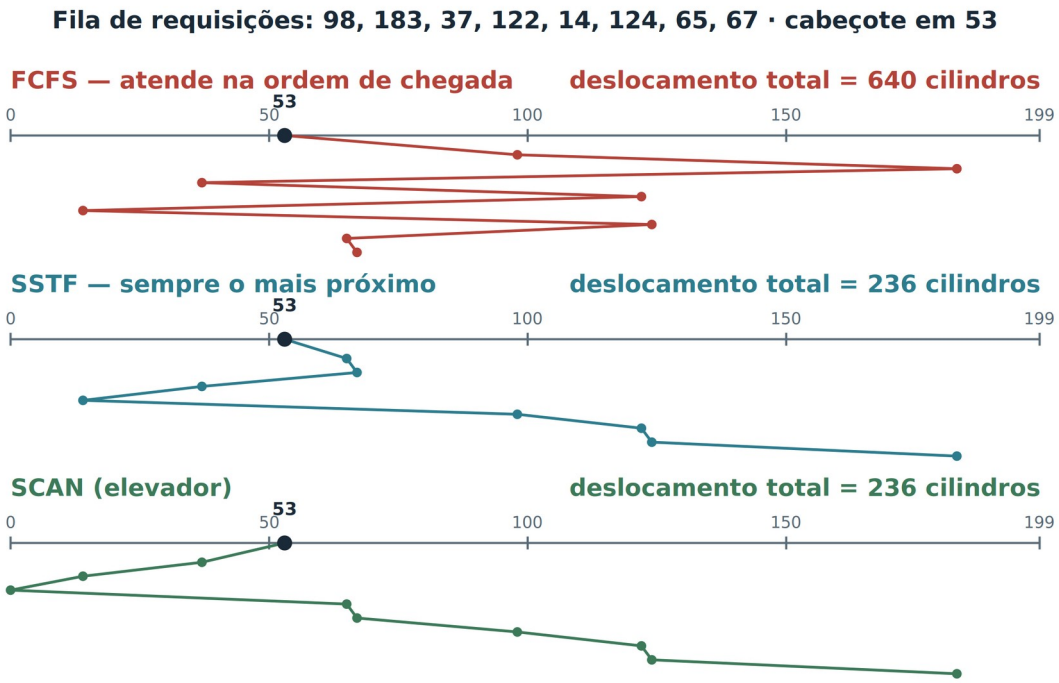


Figura 21 — FCFS, SSTF e SCAN sobre a mesma fila de requisições.

COMO LER A FIGURA

Cada uma das três linhas horizontais é o disco, do cilindro 0 ao 199, com o cabeçote partindo do cilindro 53 (ponto preto). A trajetória colorida descendente mostra a ordem em que as requisições foram atendidas: cada degrau é um movimento do cabeçote. No FCFS, a trajetória cruza o disco inteiro várias vezes; no SSTF, ela é curta e localizada; no SCAN, vai primeiro até a borda esquerda (cilindro 0) e depois varre para a direita.

O deslocamento total está à direita de cada título. SSTF e SCAN empatam em 236 neste exemplo, mas a frase de rodapé diz por que o SCAN é preferido em sistemas com carga alta: o SSTF pode deixar uma requisição distante esperando para sempre; o SCAN não.

Referencial teórico

TANENBAUM, A. S.; BOS, H. Sistemas operacionais modernos. 4. ed. São Paulo: Pearson, 2016.

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. Fundamentos de sistemas operacionais. 9. ed. Rio de Janeiro: LTC, 2018.

MACHADO, F. B.; MAIA, L. P. Arquitetura de sistemas operacionais. 5. ed. Rio de Janeiro: LTC, 2013.

STALLINGS, W. Arquitetura e organização de computadores. 10. ed. São Paulo: Pearson, 2017.

BRASIL. INEP. Exame Nacional de Desempenho dos Estudantes — provas de Ciência da Computação (bacharelado), edições de 2017 e 2021. Brasília: Inep.

ARPACI-DUSSEAU, R. H.; ARPACI-DUSSEAU, A. C. Operating Systems: Three Easy Pieces. Arpaci-Dusseau Books, 2018. Disponível em: https://pages.cs.wisc.edu/~remzi/OSTEP/ (livro-texto de acesso aberto; capítulos sobre execução direta limitada, paginação, TLB e substituição de páginas).

DIJKSTRA, E. W. The structure of the 'THE'-multiprogramming system. Communications of the ACM, v. 11, n. 5, p. 341-346, 1968. Transcrição disponível em: https://www.cs.utexas.edu/~EWD/transcriptions/EWD01xx/EWD196.html

COFFMAN, E. G.; ELPHICK, M.; SHOSHANI, A. System deadlocks. ACM Computing Surveys, v. 3, n. 2, p. 67-78, 1971.

DENNING, P. J. The working set model for program behavior. Communications of the ACM, v. 11, n. 5, p. 323-333, 1968.

BELADY, L. A. A study of replacement algorithms for a virtual-storage computer. IBM Systems Journal, v. 5, n. 2, p. 78-101, 1966.

THE LINUX KERNEL DOCUMENTATION. ext4 General Information; Journal (jbd2); EEVDF Scheduler. Disponível em: https://docs.kernel.org/

SUSE. Introduction to the boot process. SUSE Linux Enterprise Server Administration Guide. Disponível em: https://documentation.suse.com/

CORNELL UNIVERSITY. CS 4410 Operating Systems — Page replacement and thrashing (notas de aula). Disponível em: https://www.cs.cornell.edu/courses/cs4410/

Nota: todos os diagramas deste material são de elaboração dos autores, esquematizados com base nas obras acima.